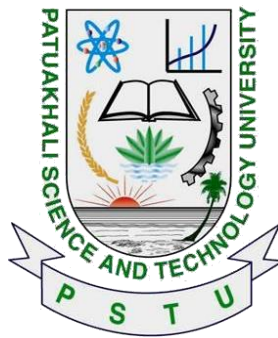


An Efficient Software Security Assurance Model to Develop Secure Institutional Information System

MS Thesis

MD. MAHMUDUL HASAN FAISAL



Department of Computer Science and Information Technology (CSIT)

Patuakhali Science and Technology University

Dumki, Patuakhali-8602, Bangladesh

June, 2024

An Efficient Software Security Assurance Model to Develop Secure Institutional Information System

A Thesis

Submitted to

Patuakhali Science and Technology University, Patuakhali

In Partial Fulfillment of the Requirements

For the Degree of Master of Science

in

Computer Science and Information Technology (CSIT)

By

MD. MAHMUDUL HASAN FAISAL

Roll/ID No. CSITJD004/2021

REG. NO. 06530



Department of Computer Science and Information Technology (CSIT)

Patuakhali Science and Technology University

Dumki, Patuakhali-8602, Bangladesh

June, 2024

An Efficient Software Security Assurance Model to Develop Secure Institutional Information System

A Thesis

Submitted to

Patuakhali Science and Technology University, Patuakhali

In Partial Fulfillment of the Requirements

For the Degree of Master of Science

in

Computer Science and Information Technology (CSIT)

By

Md. Mahmudul Hasan Faisal

Approved as to style and contents by

Professor Chinmay Bepery
Supervisor

Md. Mahbubur Rahman
Co-Supervisor

Professor Chinmay Bepery
Chairman, Defense Committee and
Department of Computer Science and Information Technology (CSIT)

May, 2024

DECLARATION

I declare that, the thesis hereby submitted by me for the Master degree at the Patuakhali Science and Technology University is my own independent work and has not previously been submitted by me at another university/faculty for any degree.

(Md. Mahmudul Hasan Faisal)

Certified by

(Professor Chinmay Bepery)

ACKNOWLEDGMENTS

At first, I am thankful to Almighty Allah who blesses us to complete this study successfully and also deliver this within the deadline. Blessings and peace be upon his beloved and faithful friend, Hazrat Muhammad (PBUH), who is a unique role model for the entire world.

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of people whose ceaseless cooperation made it possible, whose constant guidance and encouragement crown all efforts with success.

I am grateful to my spiritual guide, Alhaj Maulana Shah Muhammad Mohibullah (may Allah bless him), for his guidance in fostering my self-confidence, self-purification, and moral principles that influenced my academic journey.

I am profoundly appreciative to the ICT Division, Government of the People's Republic of Bangladesh, for awarding me a Masters level scholarship/fellowship in the 2nd round of the 2023-2024 fiscal year, which significantly contributed to the completion of this work.

I am grateful to all members associated with the Department of Computer Science and Information Technology (CSIT), Patuakhali Science and Technology University as they provide me infrastructure and facilities to implement and publish this system.

I am especially thankful to Professor Dr. Md. Samsuzzaman, Department of CCE, Md. Mahbubur Rahman, Assistant Professor, Department of CSIT, Patuakhali Science and Technology University as their support in implementing and publishing the system.

I am also grateful to my supervisor Professor Chinmay Bepery, the Chairman of the Department of CSIT, Patuakhali Science and Technology University.

With Best Regards,

Md. Mahmudul Hasan Faisal

ABSTRACT

Institutional information systems (IIS) are critical infrastructure components that manage and safeguard sensitive organizational data, including intellectual property and confidential records. Security breaches in these systems can have devastating consequences, leading to financial losses, reputational damage, and even legal repercussions. Therefore, ensuring the security of IIS throughout the software development lifecycle (SDLC) is paramount. This paper explores a multi-faceted approach to securing system applications through the implementation of robust security constraints. We examine into essential constraints encompassing authentication, authorization, data security, secure coding practices, and system hardening, emphasizing their significance in fortifying the overall security posture. Software security assurance (SSA) models provide a structured approach to achieving this objective.

This paper recommends for practicing Secured Institutional Information System Development Model (SIISDM), a novel effective SSA models for developing secure IIS. A comparative analysis shows SIISDM, integrating SDL and ISO/IEC 27001, provides a holistic security framework for IIS, unlike SDL/BSI's lifecycle focus or OWASP Top 10's vulnerability-specific approach. Successful SIISDM implementation demonstrates its effectiveness in enhancing security by proactively identifying and mitigating vulnerabilities, ensuring robust configuration management, and dynamically adjusting security, thereby reducing data breach risks and fostering stakeholder trust.

Table of Contents

ACKNOWLEDGMENTS	iv
ABSTRACT.....	v
Table of Contents.....	vi
List of Figures.....	ix
List of Tables	x
CHAPTER 1: INTRODUCTION	11
1.1 Introduction.....	11
1.2 Research Background.....	12
1.3 Problem Statement	13
1.4 Motivation.....	14
1.5 Objective of Research	14
1.5.1 Specific Objectives.....	15
1.6 Organization of the Thesis	15
CHAPTER 2: LITERATURE REVIEW	17
2.1 Introduction.....	17
2.2 Constructive Philosophy	17
2.3 Critical Review of Recent Study.....	18
2.4 Existing Security Assurance Model.....	22
CHAPTER 3: METHODOLOGY	23
3.1 Introduction.....	23
3.2 Conceptual Framework	24
3.3 Constraint Programming	25
3.3.1 Formalizing the CSP Framework	25
3.3.2 Optimization for Objective function	26
3.3.3 Key Considerations and Trade-offs.....	28
3.4 System Security Constraints	29
3.4.1 Resource Constraints and Denial-of-Service Attacks	30
3.4.2 Access Control Constraints and Permissions	30

3.4.3 Probabilistic Security Constraints and Risk Management	31
3.5 Dataset Collection	32
3.6 Critical Software Security Risks	34
3.6.1 Insecure Development Practices.....	34
3.6.2 Integration Vulnerabilities.....	35
3.6.3 Insider Threats	35
3.6.4 Data Security Challenges	36
3.6.5 Hosting Environment/Cloud Security Concerns	36
3.7 Multi-Layered Security Constraints.....	37
3.7.1 Authentication and Authorization	37
3.7.2 Data Security	38
3.7.3 Secure Coding Practices	38
3.7.4 System Hardening	38
3.8 Security Constraints for IIS Development	39
3.8.1 Environment Segregation Policy	39
3.8.2 Precedence of Environment Variables	41
3.8.3 Scope selection for DI Container	45
3.8.4 Rate Limit Configuration	47
3.8.5 Anti-forgery Implementation.....	49
3.8.6 Synergistic Approach for Authorization	51
3.9 The Secured Institutional Information System Development Model (SIISDM) ..	56
3.9.1 Building Security Awareness	57
3.9.2 Setting the Security Foundation	57
3.9.3 Threat Modeling and Mitigation	57
3.9.4 Secure Coding and Development	57
3.9.5 Multi-Layered Security Checks.....	58
3.9.6 Safe Deployment Process (SDP)	58
3.9.7 Continuous Security Monitoring and Management	59
3.10 Implementation and Testing of SIISDM.....	59
CHAPTER 4: RESULT AND DISCUSSION	61
4.1 Introduction.....	61
4.2 Security Threats and Issues.....	61
4.3 Security Assurance Model for IIS.....	62

4.4 Adaptability of SIISDM	63
4.4.1 Precedence of Environment Variables	64
4.4.2 Dynamic Rate Limit Configuration.....	65
CHAPTER 5: CONCLUSION AND FUTURE WORKS.....	65
5.1 Introduction	65
5.2 Key Contributions	65
5.3 Further Research Scopes	66
Self-Tuning Mechanisms and Threat Intelligence Integration	66
Standardization and User Experience Considerations.....	66
References.....	68
APPENDICES	73
Appendix A	73
Appendix B	75
Appendix C	76

List of Figures

Figure 1 BGD e-GOV CIRT Architecture of Application Security, 2024	12
Figure 2 Conceptual Framework of the Model Implementation	24
Figure 3 Research Work-flow for formulating the SAM	31
Figure 4 Statistical Analysis of the Survey Data	33
Figure 5 Sequence of Environments for System Development	40
Figure 6 Precedence of Environment Variables Hierarchy	44
Figure 7 Different Scopes for Dependency Injection Instances	45
Figure 8 Secured Institutional Information System Development Model (SIISDM).....	56
Figure 9 Request Timing from out of PSTU Network (Before Deploying SIISDM)	64
Figure 10 Request Timing from out of PSTU Network (After Deploying SIISDM).....	64
Figure 11 Request Timing from PSTU Network (Before Deploying SIISDM).....	65
Figure 12 Request Timing from PSTU Network (After Deploying SIISDM)	65
Figure 13 Response Header (Before Rate Limit Implementation)	66
Figure 14 Response Header (After Rate Limit Implementation)	66
Figure 15 Response Header (After API calls quota exceeded).....	66

List of Tables

Table 1 Comparison between Authorization Domains.....	54
Table 2 SIISDM Applied System Applications.....	59
Table 3 Security Issues for Developing IIS	61
Table 4 Comparison between different Security Assurance Model for IIS.....	63

CHAPTER I

INTRODUCTION

1.1 Introduction

Information System security refers to the process of creating and developing software that assures the integrity, confidentiality, and availability of its code, data, and services. By the definition, security in system development is a collection of efforts and solutions focused on the protection of products from malicious or unintentional manipulations (Khan, Khan, Khan, et al., 2022). Any commonly used digital solution today must have measures in place to prevent unauthorized access, change, or destruction of the data and code contained within it.

Software development organizations treat security as an afterthought issue, and as a result, they continue to face security threats. Incorporating security at any level of the System Development Life Cycle (SDLC) has become an urgent requirement (Umeugo, 2023). Consideration of security in the SDLC is essential to implementing and integrating a comprehensive strategy for managing risk for all information technology assets in an organization.

Software security assurance is a process that helps design and implementation of the software for protecting the resources and data contained in and controlled by the software. A security assurance model is an archetype of a framework to help organizations formulate and implement a strategy for software security that is tailored to the specific risks faced by the organization (Khan, Khan, Alzahrani, et al., 2022). The model have goals for ensuring appropriate security adequate protection for individual system components from threat vectors where the process spans the entire SDLC of a system considering all of existing or emerging threats and vulnerabilities.

The Institutional Information System refers to the establishment of an integrated platform where participate all individual roles of stakeholders of the institute and access the institutional data that have been derived, aggregated, processed, organized, structured and presented in appropriate visual and systematic representations.

Beyond this, Instituting information system security involves creating a set of policies and practices to protect your data from unauthorized access, use, disclosure, disruption, modification, or destruction.

1.2 Research Background

In today's digital landscape, system applications play a pivotal role, managing critical data and facilitating core business functions. However, this ubiquity also renders them attractive targets for cyberattacks. Security breaches can have devastating consequences, resulting in compromised confidential information, financial losses, and erosion of trust. To mitigate these risks, implementing a comprehensive security framework with well-defined constraints is paramount. This paper examines various security constraints that contribute to a robust defense strategy for system applications.

On January 2024, a comprehensive Guidelines on information security practices for public and government entities has published by BGD e-GOV CIRT that includes application security architecture (Saiful Alam Khan et al., 2024).

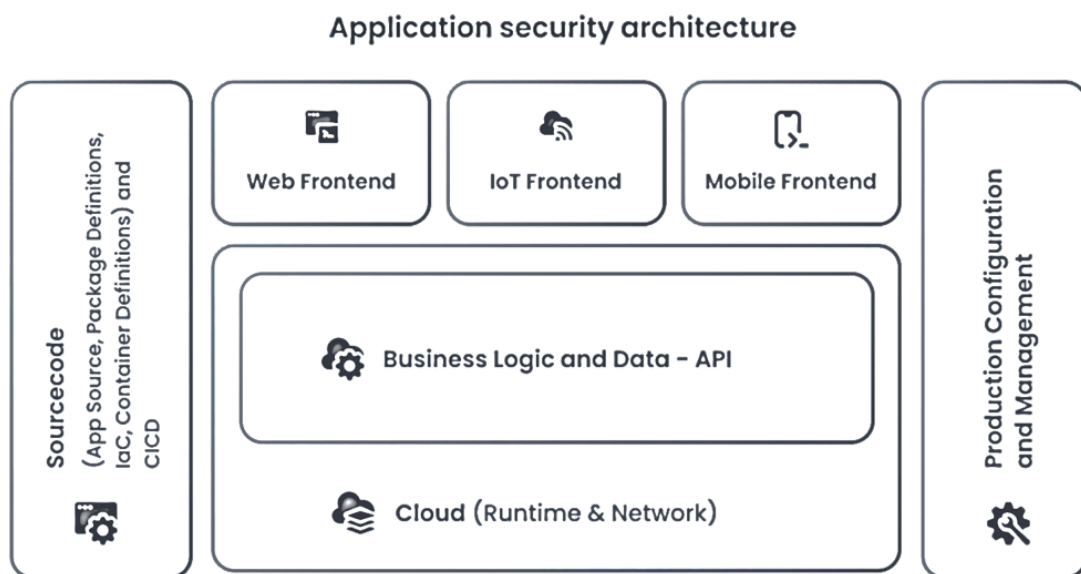


Figure 1 BGD e-GOV CIRT Architecture of Application Security, 2024

Based on the context, our proposed work introduce a new Security Assurance Model (SAM) for development of Institutional Information System that is adaptable to all contemporary scenarios, emphasizing individual institutes/organizations.

1.3 Problem Statement

According to record of CVE-2023-29552, published by US National Vulnerability Database 2023, Over 2,000 organizations were identified world-wide as having vulnerable instances. Attackers could potentially leverage these vulnerable instances to launch a cyber-attack targeting the system owners and/or other organizations. Many Fortune 1000 organizations were identified as having vulnerable instances (MITRE Corporation, 2023). Fortune 1000 impact spans many sectors, including finance, insurance, technology, telecommunications, manufacturing, healthcare, hospitality, and transportation. Like this, the threats can potentially impact business continuity and result in financial loss, even if an attacker has limited resources.

According to the report of Bangladesh Cyber Threat Landscape published on 2022, threat actors and cybercriminals are always sneaking in for vulnerable services which can be exploited to gain initial access to a targeted infrastructure. Later on, this exposure may lead to ‘privilege escalation’ and ‘lateral movement’ to facilitate the intruder to setup command and control communication, steal data or execute ransomware attack by encrypting data (Barkatullah et al., 2023). Threat intelligence unit of BGD e-GOV CIRT states that the organizations, including CIIs, government and private sector, are being exposed with numerous vulnerable services which are lucrative for cyber criminals.

To overcome these security issues, organizations must implement appropriate security measures to safeguard their networks and servers from being used in such attacks. It is equally important to enforce strong authentication and access controls, allowing only authorized users to access the correct system resources. Implementing strong security measures and access controls can reduce the risk of falling victim or unwillingly participating in these types of attacks, while incident response plans can mitigate the effects of such an attack.

From the context of study, the findings has indicates the necessity for identification of security threats to information system that institutes/organizations should avoid during the system development. Within this, there is a huge challenge to build awareness and adapt the mitigation practices for institutional organizations can use to upgrade existing solutions to secure information systems. From these issues, there is a potential field to contribute by introducing an efficient software security assurance model which is capable of assisting individual institutes/organizations in determining their readiness to develop secure institutional information system.

1.4 Motivation

During my undergraduate study, we have developed and deployed several system applications for our institute, whose are integrated in PSTU Institute Information System (PIIS). From 2021, after deploying those in institute's own infrastructure maintaining standard terms and policies for production, we have faced a several medium and severe attacks including DoS, Brute-Force, Ransomware intrusion and so on. Although, we don't have any remarkable incident yet by those attacks because we as maximum as possible security concern during the development phases. From the continuous analysis of critical software risks and prevention mechanism, we adapt mitigation procedures within the policy and expert guidelines. Rather than using wrap-up service purchasing, we always preferred solution by mitigating foundational security issues. Within the period, we realized that, we have to do study on this domain to build-up a security assurance model that can assist same type of public institutes by assessing their readiness to develop secure institutional information system.

According to Smart Bangladesh Vision 2041, the target is refers to inclusive digital transformation to build a developed and prosperous country by 2041 (Pal & Sarker, 2023). Thus our research work strongly motivated from the essence of contribution in national cyber threat intelligence and incident research relevant to ICT and assist the implementation of Smart Bangladesh Vision 2041 inclusive digital transformation.

1.5 Objective of Research

The digital age has brought immense benefits to institutions, but it has also introduced new security challenges. Sensitive data residing within information systems is vulnerable to unauthorized access, manipulation, and disruption. To address these concerns, this paper outlines a multi-faceted approach for developing secure institutional information systems.

The first step towards building secure systems necessitates a comprehensive understanding of potential threats. This involves conducting a thorough risk assessment to identify vulnerabilities within the software, network infrastructure, and institutional policies. Common threats include unauthorized access attempts, malware infections, and data breaches. By proactively identifying these vulnerabilities, institutions can prioritize mitigation efforts and focus resources on areas of greatest risk.

Beyond threat identification, a robust software security assurance model is crucial. Hence a lot of services that can be wrap-up on an information system software after deployment

but these do not mitigate the vulnerability addressed within the system itself. So the consideration of security measures should be applied during the development period within the system design core implementation. Our principal target is to develop software security assurance model capable of assisting institutes/organizations in determining their readiness to develop secure information system. Such a model would serve as a standardized framework for evaluating an institution's readiness for secure information system development. The model could assess current security practices, identify gaps in existing controls, and recommend best practices for improvement. This comprehensive evaluation ensures a secure foundation is established before embarking on system development.

Security should not be an afterthought; it should be an integral part of the entire information system lifecycle. This objective necessitates integrating the security assurance model into ongoing system operations. The model can be used for continuous monitoring of vulnerabilities, ensuring that new features and functionalities do not introduce unintended security risks. Additionally, regular penetration testing and security audits can be employed to proactively identify and address potential weaknesses.

In other word, this paper presents a multi-faceted approach for developing secure institutional information systems.

1.5.1 Specific Objectives

The objectives of this research study are:

- To identify the security threats and issues for development of secure institutional information system.
- To develop software security assurance model capable of assisting institutes/organizations in determining their readiness to develop secure information system.
- To apply the model in a real-time information system for enhancing adoptability with existing security policies.

1.6 Organization of the Thesis

According to documentation guideline provided by the Postgraduate studies manual this thesis report arranged in standard style and format. The thesis comprises of five chapters. The thesis organization is generally described as follows:

The first chapter presents the inception of the thesis. The problem and research motivation are described in this chapter. The research objectives of the thesis are also outlined.

The second chapter includes the vital part of the study background. This describe the critical review on research publications, limitation or future scopes for existing technologies on the research domain and the findings from other research works.

The third chapter describes the approaches to complete the research study including the conceptual framework, architectural overview and implementation scopes in detail.

The fourth chapter describes the experimental fields, the comparison of pre-and post-implementation consideration with experimental result and discussion.

The last chapter concludes the key contribution of this study and future scopes of the work in general.

CHAPTER II

LITERATURE REVIEW

2.1 Introduction

The ever-growing reliance on information technology necessitates a robust approach to ensuring the security of critical systems. Security assurance models (SAMs) have emerged as a prominent tool for evaluating and enhancing the security posture of software and information systems. This literature review delves into the existing body of research on SAMs, exploring their core principles, functionalities, and limitations.

2.2 Constructive Philosophy

The ever-expanding realm of information technology necessitates robust security measures to safeguard sensitive data and systems. While the Quran, the foundational complete code of life, predates modern information systems, it offers a wealth of ethical principles that can be applied to contemporary information security practices. The Quran emphasizes the importance of fulfilling trusts and safeguarding entrusted possessions (Quran, The Holy Quran, 23:8). In the context of information security, this principle translates to protecting the confidentiality of sensitive data. Organizations must implement robust access controls, encryption strategies, and data loss prevention mechanisms to ensure that only authorized individuals can access confidential information. The Quran underscores the concept of accountability, reminding individuals that they will be held responsible for their actions in the afterlife (Quran, The Holy Quran, 6:164). Within information security, this principle translates to establishing clear ownership and accountability for data handling practices. Organizations should implement role-based access control (RBAC) and maintain detailed audit logs to track user activity and data access. This fosters a culture of responsibility and deters malicious behavior. The Quran promotes uprightness and integrity in all aspects of life (Quran, The Holy Quran, 2:143). In information security, this principle translates to ensuring the accuracy and completeness of data and information systems. Organizations should implement data integrity checks, employ digital signatures, and maintain regular backups to safeguard data from unauthorized modification or loss. Additionally, vulnerability management programs and system hardening practices help maintain the overall integrity of information systems.

While the Quran does not explicitly address information security, the principles of confidentiality, accountability, and integrity offer a valuable framework for building a secure information environment. By integrating these ethical considerations into information security policies and procedures, organizations can foster a culture of responsible data handling and enhance the overall security posture of their information systems.

2.3 Critical Review of Recent Study

According to recent research study insights into software testing performed by (Gurcan et al., 2022) indicate an increasing trend for the topics “Security Vulnerability”, “Open Source”, and “Mobile Application”. Security is one of the most critical aspects of software quality.

(Khalil et al., 2023) described that, several methodologies, strategies, and models have been proposed and developed to address software security, but only a few of them give reliable evidence for creating secure software applications. Software security issues, on the other hand, have not been adequately addressed, and integrating security procedures into the SDLC remains a challenge.

According to a recent systematic literature review (SLR) was provided by (Khan, Khan, Khan, et al., 2022) to classify important studies where identified 145 security risks and 424 best practices that help software development organizations to manage the security in each phase of the SDLC.

In the proposed Security Assurance Model of Software Development for Global Software Development Vendors by (Khan, Khan, Alzahrani, et al., 2022) states that, successful integration of these activities minimizing effort, time, and budget while delivering secure software applications. A branch of new emerging software development models are required that assist in developing software that is secure by default which will raise the developer’s awareness of secure development practices.

In a qualitative research work contributed by (Alnaseef et al., 2023) they summarize that, organizations invest considerable sums of money in protecting their assets. Despite investing in secure infrastructure, organizations remain prone to security risks and cyber-attacks that exploit security flaws. Many factors contribute to the challenges related to software security, e.g., the exponential increase in Internet-enabled applications, threats from hackers, and the susceptibility of inexperienced Internet users. Moreover,

organizations tend to procure off-the-shelf software from third-party suppliers. However, gaining a complete understanding of ways to assess suppliers' readiness to provide secure software before selecting a supplier is imperative. Beyond this, the best practices undertaken by organizations in terms of acquiring secure software, which depends on six core security knowledge areas: confidentiality, integrity, availability, authorization, authentication, and accountability.

As concluded by (Spiekermann et al., 2022), with the rapid digitalization of all areas of life and work, designing Information System for well-being has become increasingly important. However, in this context, Information System should be seen as a double-edged sword: they can have positive as well as negative impacts on individual well-being.

Before this, (Spiekermann, 2021) mentioned that, for developing information system, IEEE 7000™ standard has been called Value-based Engineering, provides engineers with a clear system design and development framework; or in other words, an ethical SDLC.

While mandatory security standards are salient in the management of information security, the related theoretic studies are scarce, especially when strategic hackers are considered. As the context of (Gao et al., 2022), using a game-theoretic model, this paper examines the strategic interaction in a resource sharing environment between two firms who invest in information security subject to the mandatory standard and one hacker who exerts attack efforts against the firms. It shows that the strict mandatory standard doesn't always benefit each firm even though its information systems can be better protected. As the firms share more resource, each firm lacks strong motivation to invest enough in information security, and as a result stricter security standard should be formulated from the socially optimal standpoint.

Security Education Training and Awareness plays a dynamic role for organizations in endorsing resources' thoughtfulness and accessibility. This paper determines the importance of security awareness training in dealing with cyber threats. The research work proposed by (Dash & Ansari, 2022), uses the Technology Acceptance Model (TAM), indicating that at-risk employees' behaviour and information security awareness training execution are successful interventions. Yet, those investigations did not examine Artificial Intelligence (AI) enabled training, so this study fills that literature gap. This analysis used a qualitative research design. The article examines employees' behaviour and the effectiveness of AI-based security awareness training programs. The study here

helps analyse information security awareness training in the workplace, encouraging behavioural transformations that would restrain data breaches by incorporating the users' exposure and the stringency of coercion and the retort to peril in prophesying behaviour discretions.

Improving the quality assurance processes and acquiring accreditation are top priorities for academic programs. The learning outcomes assessment and continuous quality improvement represent core components of the quality assurance system (QAS). Current assessment methods suffer deficiencies related to accuracy and reliability, and they lack well-organized processes for continuous improvement planning. Moreover, the absence of automation, and integration in QA processes forms a major obstacle towards developing efficient quality system. There is a pressing need to adopt security protocols that provide required security services to safeguard the valuable information processed by QAS as well. (Alkhatib, 2022) proposed an effective methodology for learning outcomes assessment and continuous improvement processes. The current study serves scientific programs struggling to achieve academic accreditation, and gives rise to fully automating and integrating the QA processes and adopting modern AI and security technologies to develop effective QAS.

(Umeugo, 2023) introduce a new secure software development lifecycle (SSDLC) developed based on Secure Software Principles that focused on a case study for adoption in software SMEs.

In the analysis of multiple security practices and their aggregate score affect the security outcome of software, (Zahan et al., 2023) proposed model to identify important security practices to enhance software security and evaluate the model using open source datasets. Thus indicates that, there required collecting more granular-level data for security practices. There also need more large-scale focus studies to identify accurate vulnerability data in software components for modeling and understand how security practices impact security outcomes.

To explore the role of assurance context in system security assurance evaluation, a conceptual model has proposed (Wen & Katt, 2023, 2024). Thus highlights the interrelationships between the various elements of the assurance context, including system boundaries, stakeholders, security concerns, regulatory compliance and assurance assumptions and regulatory compliance. There is a pressing need for comprehensive

model validation and refinement across diverse industries and organizational scales. They concluded that, to explore the development of automated tools or software solutions that can assist evaluators in integrating the assurance context into their evaluations.

To enhance Security Assurance in Software Development an AI-Based Vulnerable Code Detection with Static Analysis has proposed (Rajapaksha et al., 2024). Several ML classifiers were tested on a feature vector generated using Natural Language Processing techniques. The Random Forest and Extreme Gradient Boosting classifiers were found to perform well in binary and multi-class approaches, respectively. However, in real-time, reliability issues of AI indicates that, the ML models are not appropriate to security practice in production environment.

Constraint Programming (CP) has emerged as a powerful paradigm for tackling large-scale combinatorial optimization problems. Its strength lies in its ability to efficiently locate high-quality feasible solutions within reasonable timeframes.

Resource constraints of cyber-physical systems make tailoring a security solutions to them challenging. (Raiyat Aliabadi et al., 2021) formulated the problem of constructing an intrusion detection system for cyber-physical systems as an optimization problem of constraint programming.

Model-Based System Engineering (MBSE) has become a cornerstone for designing complex systems. However, efficiently managing design constraints within the MBSE framework remains a challenge. A recent work by (Pierre-Alain & Laurent, 2021) propose a novel approach that merges MBSE with Constraint Programming (CP) to address this challenge. The key strength of the paper lies in its introduction of a unique integration between MBSE and CP. MBSE offers a structured way to represent system models, while CP excels at solving problems with intricate constraints. This integration has the potential to streamline the system design process, particularly for problems involving numerous design constraints.

By critically reviewing the existing literature, this investigation aims to provide a comprehensive understanding of SAMs and their efficacy in bolstering the security of information systems. The review can be valuable for researchers, practitioners, and policymakers seeking to develop and implement effective strategies for building trustworthy and resilient systems in the digital age.

2.4 Existing Security Assurance Model

Several well-established SSA models offer a framework for integrating security throughout the SDLC:

- **Secure Development Lifecycle (SDL):** Developed by Microsoft, the SDL emphasizes proactive security measures embedded throughout the development process (Gluck & Mazzoli, 2024). It outlines a series of activities, from threat modeling and secure coding practices to vulnerability testing and penetration testing, ensuring a holistic approach to securing software.
- **Building Security In (BSI):** Created by Cigital, BSI focuses on integrating security considerations into every phase of the SDLC (McGraw, 2006). It emphasizes secure design principles, secure coding practices, and ongoing security verification and validation activities.
- **Open Web Application Security Project (OWASP) Top 10:** This widely recognized industry standard provides a list of the ten most critical web application security risks (Aljabri et al., 2022; Wen & Katt, 2023). By focusing on mitigating these vulnerabilities during development, institutions can significantly enhance the security posture of their IIS.

These models provide a robust foundation upon which to build secure IIS. However, successful implementation necessitates additional considerations.

Within the context, this research work introduce a new Security Assurance Model (SAM) for development of Institutional Information System that is adaptable to all contemporary scenarios, emphasizing individual institutes/organizations.

CHAPTER III

METHODOLOGY

3.1 Introduction

Security assurance models offer a structured and comprehensive methodology for assessing and fortifying the security posture of information systems. Understanding their core principles, functionalities, and limitations is essential for researchers, practitioners, and policymakers seeking to develop and implement effective strategies for building trustworthy and resilient institutional information systems in the digital age.

Prescriptive analytics is the most complex type of analytics. It combines internal data, external sources, and machine-learning techniques to provide the most effective outcomes. In prescriptive analytics, a decision-making process is applied to descriptive and predictive models to find the combinations of existing conditions and possible decisions that are likely to have the most effect in the future. This process is both complex and resource intensive but, when done well, can provide immense value to an organisation (Lepeniotti et al., 2020).

As the most complex form of analytics, prescriptive analytics not only pose technical challenges, but are also influenced by external factors such as government regulation, market risk, and existing organisational behaviour (Frazzetto et al., 2019).

As well as identifying and programming each decision, data scientists developing prescriptive models need to prevent missteps by ensuring that all possible outcomes are considered. After deploying these systems they must test their models repeatedly, to ensure they are making meaningful recommendations and there's no risk of costly mistakes.

In addition, constraint programming is a promising approach to this direction. Constraint programming is a programming paradigm wherein relations between variables are stated in the form of constraints (Ceberio & Kreinovich, 2014). While the general case of constraint satisfaction problems is NP-complete, the expressiveness, propagation techniques, and domain-specific knowledge integration capabilities offered by CP render it a valuable tool for tackling complex optimization problems in various domains.

3.2 Conceptual Framework

By the problem specification, the research study formed in a qualitative analysis. Target to developing model will be tended to inductive research work which includes steps such as observation, pattern recognition, development of theory/model etc. The testing performance of the model will be tended to deductive work which includes steps such as formulation of a hypothesis based on model, feasibility of test study, collecting data from existing system, analysis of data, drawing conclusions and address future scopes.

It is necessary to conduct sufficient systematic literature review, mainly focusing on empirically validated solutions, to identify software security risks and strategies for managing these risks at each stage of the software development lifecycle (SDLC). Based on criteria for determining the importance of the stated information system security risk, we will collect proper datasets from the secure software development professionals and other reliable sources. These dataset will be fed to established prescriptive models for estimating the quantification of security risks into critical software security risks that's are frequently identified at production in cause of unaware gap in SDLC practice during development of the system. The result will be evaluated frequently unless it reach to reasonable accuracy through the identification of potential fixes for the lack of proper attention to security issues during the design phase. The stable evaluated model will be applied to fresh data to monitor outcome.

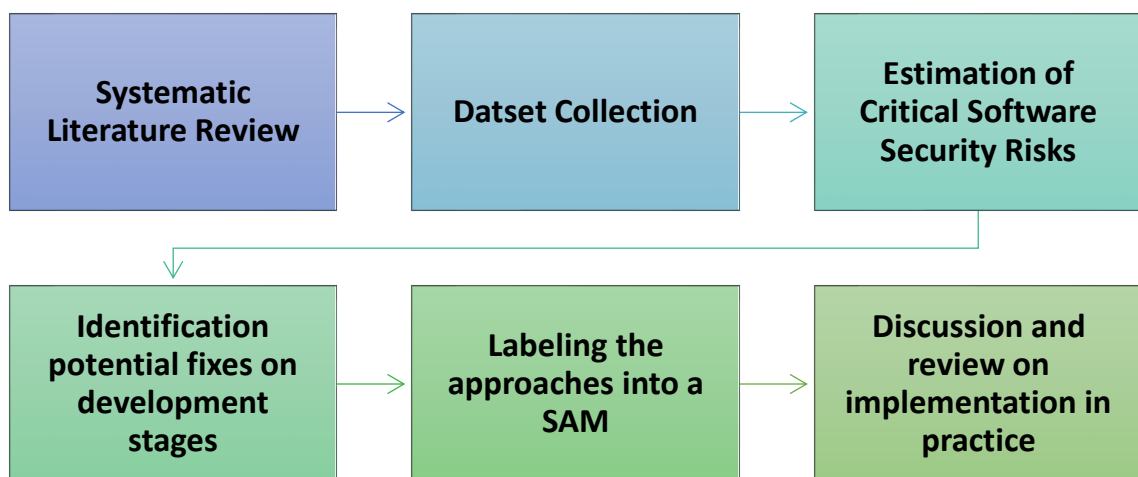


Figure 2 Conceptual Framework of the Model Implementation

To introduce sustainable solution to determining the organization readiness to develop information system, the approaches should be specify in a software security assurance

model. Apply comparative practice with existing models to establish this model for better performance and ensure better security and privacy.

Prescriptive analytics, the most complex form of data analysis, leverages internal data, external sources, and machine learning to recommend the most impactful actions. This involves applying decision-making processes to descriptive and predictive models, identifying the most effective combinations of current conditions and potential decisions for the future. While resource-intensive and technically challenging, successful prescriptive analytics can provide immense value to organizations. However, its effectiveness is influenced by not only technical hurdles but also external factors like government regulations, market risks, and established organizational behavior.

3.3 Constraint Programming

Constraint programming (CP) offers a powerful paradigm for solving problems by modeling them through variables and constraints. Unlike traditional programming approaches that focus on algorithms, CP prioritizes the relationships between variables and the restrictions governing their values. While these constraints can be expressed in various forms, a mathematical framework provides a foundation for understanding and analyzing CSPs, a core concept in CP (Pierre-Alain & Laurent, 2021).

Here we explore the mathematical representation of Constraint Satisfaction Problems (CSPs) within the context of CP.

3.3.1 Formalizing the CSP Framework

A CSP can be mathematically represented by a three-tuple:

Set of Variables: This set encompasses all variables involved in the problem. Each variable represents an unknown value to be determined.

$$X = \{x_1, x_2, \dots, x_n\} \dots\dots\dots (1.1)$$

Set of Domains: This set associates a domain (D_i) with each variable (x_i). A domain defines the set of permissible values that the corresponding variable can take on.

$$D = \{D_1, D_2, \dots, D_n\} \dots\dots\dots (1.2)$$

For instance, the domain for a variable representing age might be $D_{\text{age}} = \{1, 2, \dots, 100\}$.

Set of Constraints): This set defines the relationships or restrictions that govern the variables. Constraints limit the possible combinations of variable assignments that satisfy the problem conditions.

$$C = \{c1, c2, ..., cm\} \dots\dots\dots (1.3)$$

Constraints can be expressed in various mathematical forms:

- **Equality Constraints ($x1 = x2$):** This constraint mandates that variables $x1$ and $x2$ possess the same value.
- **Inequality Constraints ($x1 < x2$):** This constraint dictates that the value of $x1$ must be less than the value of $x2$.
- **Logical Constraints (NOT ($x1 = x2$)):** This constraint prohibits variables $x1$ and $x2$ from having the same value.
- **Membership Constraints ($x1 \in \{a, b, c\}$):** This constraint specifies that the value of $x1$ must be one of the enumerated options (a, b, or c) within the set.

A solution to a CSP is an assignment of values to each variable in X such that all constraints in C are satisfied. In essence, the objective of CP is to identify such an assignment. The mathematical representation provides a formal framework for analyzing the solvability of a CSP and the development of algorithms to search for solutions.

It's crucial to acknowledge that the mathematical representation outlined above serves as a foundational framework. Constraint programming offers the flexibility to express constraints using various functionalities and data structures that extend beyond simple equations. This empowers the modeling of complex relationships and real-world problems that cannot be neatly captured by a single set of equations.

3.3.2 Optimization for Objective function

Constraint Programming (CP) has emerged as a powerful technique for tackling combinatorial optimization problems. Unlike traditional mathematical programming approaches that rely on a single, all-encompassing objective function, CP prioritizes finding solutions that satisfy a set of imposed constraints. However, this doesn't negate the importance of optimization goals within CP.

In other words, finding an optimal solution, or even any feasible solution in the case of some highly constrained problems, is computationally intractable for large problem instances.

In constraint programming, the design of the program is dictated by specific constraints determined prior to development. After a problem is identified, the initial constraints are defined and condensed into the current program constraints. As solutions are developed and tested, decisions are made as to whether the constraints must be modified or retracted to remove duplications or whether new ones must be developed. The process cycles through current restraints and solutions until a solution is developed that meets all the constraints and solves the initial problem (Varela-Vaca & Gasca, 2013).

CP focuses on finding assignments to a set of variables (X) such that all defined constraints (C) are fulfilled. These constraints express relationships between variables, encompassing equalities, inequalities, logical statements, and more. The core objective of CP is to identify a solution (variable assignment) that adheres to all constraints, ensuring problem feasibility. While constraint satisfaction forms the foundation of CP, incorporating optimization goals allows for finding not just any feasible solution, but the "best" solution according to a specific criterion. Here, we explore two primary approaches to represent and integrate optimization goals within CP:

1. Cost Functions and Constraint Prioritization

Although CP doesn't have a single objective function, it can be defined cost functions associated with individual constraints. These cost functions assign a numerical value (cost) to violations of the corresponding constraint. The CP solver then prioritizes finding solutions that minimize the total cost of constraint violations.

Mathematically, this concept can be represented as:

$$\text{Minimize } \sum w_i * \text{cost}(c_i) \quad \dots\dots\dots (2.1)$$

Where,

- $\sum$ denotes summation over all constraints (c_i).
- w_i signifies a weight associated with each constraint, reflecting its relative importance in the optimization process.

- ***cost(c_i)*** represents the cost function for a specific constraint, returning a numerical value based on the degree of violation.

By minimizing the total cost of constraint violations, the solver prioritizes solutions that adhere to the most critical constraints and minimize overall constraint satisfaction "gaps."

2. Transformation to Integer Linear Programming (ILP)

In some scenarios, it's possible to reformulate the CP problem as an Integer Linear Programming (ILP) problem. ILP involves maximizing or minimizing a linear objective function subject to linear constraints. This allows you to leverage existing ILP solvers to find optimal solutions.

Here's a general form of an ILP:

$$\text{Maximize/Minimize } (\sum c_i * x_i) \quad \dots\dots\dots (2.2)$$

- ***Maximize/Minimize*** indicates whether you want to maximize or minimize the objective value.
- $\sum$ represents summation over all variables (x_i) involved in the problem.
- c_i denotes the coefficient associated with each variable, reflecting its contribution to the overall objective value.
- x_i represents the variables involved in the problem.

The constraints from the original CP problem would then be translated into linear inequalities or equalities within the ILP model. This approach leverages the strengths of ILP solvers for optimization while maintaining the flexibility of CP for constraint modeling.

3.3.3 Key Considerations and Trade-offs

- **Flexibility vs. Efficiency:** CP offers greater flexibility for modeling complex relationships and non-linear constraints, but incorporating optimization goals can introduce additional complexity.
- **Problem Reformulation:** Transforming a CP problem into an ILP model might require significant effort and may not always be straightforward.

- **Cost Function Design:** Defining effective cost functions for constraint prioritization requires careful consideration of the specific problem and desired optimization goals.

Constraint programming excels at finding solutions that satisfy complex sets of constraints. While it doesn't inherently rely on a single objective function, cost functions and transformation to ILP models provide powerful mechanisms for incorporating optimization goals within the CP framework. The choice of approach depends on the problem structure, desired optimization criteria, and the trade-off between flexibility and computational efficiency (Meng et al., 2020).

In conclusion, the mathematical representation provides a valuable lens for understanding the core principles of constraint satisfaction problems within the realm of constraint programming. This framework facilitates the analysis of problem solvability and the development of solution-finding algorithms. However, it's essential to recognize the broader capabilities of constraint programming, which extend beyond the limitations of simple mathematical equations, allowing for the modeling of intricate relationships and the solution of diverse real-world problems.

3.4 System Security Constraints

System security constraints are a cornerstone of information security, dictating permissible behaviors and limitations within a system to safeguard its resources and data. While a single, universal mathematical equation cannot encompass the vast spectrum of security constraints, various formulations can be employed to model specific security considerations. This paper explores the application of mathematical frameworks for analyzing system security constraints.

Cryptography, a cornerstone of information security, leverages concepts from information theory, particularly entropy (H), to quantify the secrecy or randomness of data. Entropy represents the uncertainty associated with a random variable. A higher entropy signifies a more secure system, as it becomes more challenging to predict the value of the variable (e.g., a cryptographic key). This relationship can be mathematically expressed as:

$$H(X) = - \sum p(x) * \log_2(p(x)) \dots\dots\dots (3.1)$$

where:

- $H(X)$ represents the entropy of the random variable X .

- $p(\mathbf{x})$ signifies the probability of a specific value $(\mathbf{x})$ occurring within $\mathbf{X}$.

By maximizing the entropy of cryptographic keys and other sensitive data, the system enhances its security posture.

3.4.1 Resource Constraints and Denial-of-Service Attacks

Security constraints can be implemented to limit resource utilization and mitigate denial-of-service (DoS) attacks. For instance, a constraint might restrict the number of login attempts per user per hour (n) to thwart brute-force attacks. Mathematically, this can be represented as:

$$n \leq N \quad \dots\dots\dots (3.2)$$

where:

- n represents the number of login attempts made by a user within a specific time frame.
- N signifies the maximum allowable number of login attempts per user within that time frame.

This constraint ensures that resource consumption for authentication purposes remains within acceptable boundaries, thereby protecting against DoS attacks.

3.4.2 Access Control Constraints and Permissions

Access control constraints define who can access specific resources $(\mathbf{R})$ within a system. These constraints can be formulated using mathematical logic or set theory. An example using set theory is:

$$U \subseteq A(\mathbf{R}) \quad \dots\dots\dots (3.3)$$

where:

- U represents the set of authorized users within the system.
- $A(\mathbf{R})$ denotes an access control function that maps users to the resources they are authorized to access $(\mathbf{R})$.

This formulation ensures that only authorized users can access specific resources, upholding data integrity and confidentiality.

3.4.3 Probabilistic Security Constraints and Risk Management

Probabilistic security constraints express the likelihood (p) of a security breach occurring. These constraints consider factors such as attack frequency, vulnerability severity, and the effectiveness of implemented security controls. They can be mathematically represented as:

$$p(\text{breach}) \leq p_{\max} \quad \dots\dots\dots (3.4)$$

where:

- $p(\text{breach})$ denotes the probability of a security breach occurring within the system.
- $p_{\max}$ signifies the maximum tolerable probability of a security breach, defined by risk management policies.

By establishing a threshold for acceptable breach probability, this constraint helps prioritize security measures and resource allocation for risk mitigation (Ponsard et al., 2020; Varela-Vaca & Gasca, 2013). The specific mathematical representation depends on the security concern being addressed. However, this analytical approach strengthens security posture by enabling the quantification of security measures and facilitating the optimization of resource allocation for achieving a desired level of security within a system.

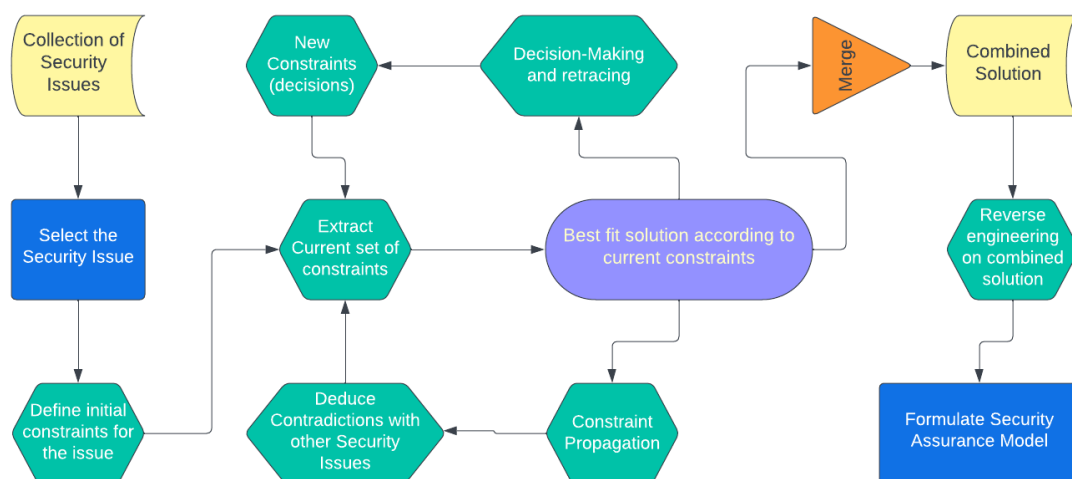


Figure 3 Research Work-flow for formulating the SAM

3.5 Dataset Collection

To accomplish the actual system security practices in institutional information system, there required conducting a survey over public organizations like educational institute, government agency and private companies.

Thus, a survey on security practice within institutional information system has conducted over 40+ public institutes of the southeast Asia on November, 2023. The Response collected over concerned IT personnel associated with Information System development/deployment/maintenance. The response data have statistically analyzed. The feedback evolution of the survey denotes potential challenges faces in implementing and maintaining the security practice within institutional information system such as the lack of guidance, awareness, experience, training and support, resources, budget and equipment.

The survey attracted a diverse range of participants, with educational institutions leading the way at 75% of respondents. Government agencies and private companies contributed equally (12.5% each), while the remaining 12.5% came from other types of organizations. This mix provides valuable insights into security practices across various institutional structures.

A positive finding is that 87.5% of institutions reported having formal guidelines in place to ensure system security. This demonstrates a commitment to establishing a security framework. However, a critical gap lies in employee and student awareness. Only 37.5% of respondents indicated that their staff or students had participated in regular training programs on software security. This highlights a need for increased focus on security education and awareness campaigns to bridge this gap.

The survey identified phishing attacks as the most common software security threat faced by institutions, followed closely by malware infections and unauthorized access attempts. These findings emphasize the importance of robust defenses against these prevalent threats. To combat these risks, institutions primarily rely on firewalls (75%) as a security measure. Antivirus/antimalware software (75%) and intrusion detection/prevention systems (37.5%) are also widely used, indicating a multi-layered approach to security.

The survey pinpointed several key challenges that institutions grapple with in securing their information systems. The most prominent challenge, identified by 50% of respondents, is the lack of awareness and training among staff regarding information

security best practices. This highlights the need for more comprehensive training programs. Additionally, insufficient budget or resources for security measures (37.5%) emerged as another significant hurdle. Keeping pace with the ever-evolving landscape of cyber threats was also a challenge reported by half of the institutions. Furthermore, limited expertise in cybersecurity (37.5%) and inadequate implementation or enforcement of existing training security policies (50%) were identified as obstacles.

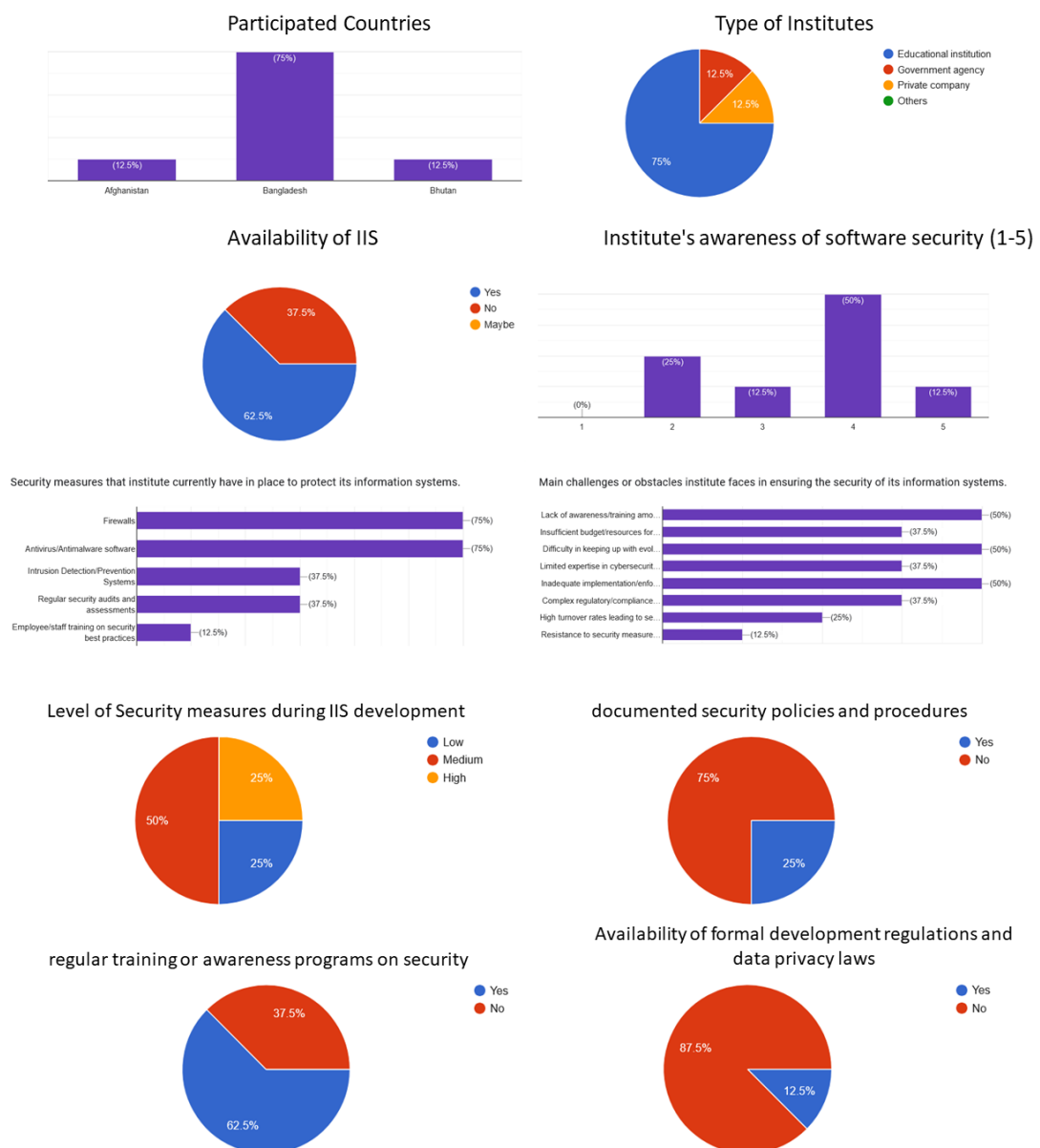


Figure 4 Statistical Analysis of the Survey Data

Despite these challenges, the survey revealed a promising trend in overall security awareness. A significant portion of institutions (25%) rated their awareness of software

security threats as high or very high. This suggests a growing recognition of the importance of information security.

To analyze system attacks and critical security risk that faced in live production servers, we have been collecting System Logs from live stand-alone production server of PSTU IIS form September 2023.

3.6 Critical Software Security Risks

In this stage, it is time to explore the multifaceted security challenges inherent in institute IS development. We begin by examining the weaknesses introduced by insecure development practices. The importance of robust coding standards, rigorous authentication/authorization mechanisms, and comprehensive security testing throughout the software development lifecycle (SDLC) will be emphasized. Next, we investigate security risks associated with system integration, focusing on vulnerabilities stemming from third-party applications and the inherent challenges of legacy systems.

3.6.1 Insecure Development Practices

Software development methodologies play a critical role in the overall security posture of any system. Insecure development practices, however, introduce fundamental vulnerabilities that can be readily exploited by malicious actors. This section explores three common shortcomings in secure development:

- **Weak Coding Standards:** The absence of robust coding standards can lead to inconsistencies and errors within the codebase. These inconsistencies can introduce exploitable vulnerabilities, similar to structural weaknesses in a building design (Bauer et al., 2023). Enforcing secure coding practices and utilizing static code analysis tools can mitigate this risk.
- **Poor Authentication and Authorization:** Ineffective authentication mechanisms resemble unlocked doors, allowing unauthorized users to gain access to a system. Similarly, weak authorization controls are akin to granting unrestricted access within a building, enabling unauthorized users to perform unauthorized actions (Kizza, 2024). Multi-factor authentication and role-based access control (RBAC) are essential security measures to address these concerns.
- **Lack of Security Testing:** Security testing is analogous to conducting safety checks on a critical system before deployment. Neglecting security testing exposes a system to undetected vulnerabilities, just as an untested car poses a

safety hazard (Pargaonkar, 2023). Regular penetration testing and vulnerability assessments are crucial for identifying and remediating security weaknesses.

These insecure development practices establish a foundation for potential compromise, highlighting the importance of prioritizing secure coding practices and rigorous security testing throughout the software development lifecycle (SDLC).

3.6.2 Integration Vulnerabilities

Modern systems often integrate various components, creating an interconnected ecosystem. However, these integrations can introduce security vulnerabilities if not managed effectively. Here, we explore two common sources of integration-related risk:

- **Third-Party Applications:** Third-party applications are akin to external vendors providing access to a building. Vulnerabilities within these applications can create a backdoor for attackers to infiltrate the main system, just as a compromised vendor can provide access to a building (Keskin et al., 2021). Thorough security assessments and ongoing monitoring of third-party applications are essential for mitigating this risk.
- **Legacy Systems:** Legacy systems resemble aging infrastructure within a building. Often designed with outdated security practices, these systems can harbor exploitable vulnerabilities due to their inherent lack of modern security features (Alnaseef et al., 2023). While complete replacement may not always be feasible, implementing compensating controls and prioritizing migration efforts can help reduce the risk associated with legacy systems.

By acknowledging these integration vulnerabilities and implementing appropriate safeguards, organizations can strengthen the overall security posture of their interconnected systems.

3.6.3 Insider Threats

Insider threats originate from individuals with authorized access to a system, making them a particularly challenging security concern. There are two primary categories of insider threats:

- **Malicious Insiders:** Malicious insiders are akin to disgruntled employees with the intent to cause harm. Their motivations can range from financial gain to revenge, and their actions can involve data theft, system sabotage, or manipulation of sensitive information (Al-Harrasi et al., 2023). Implementing strong access

controls, monitoring user activity, and fostering a culture of security awareness can help deter malicious insider activity.

- **Accidental Insider Errors:** Accidental insider errors resemble unintentional mistakes made by employees. These mistakes can involve sharing sensitive information inadvertently, clicking on malicious links, or falling victim to social engineering attacks (Reveraert et al., 2024). Security awareness training programs and implementing data loss prevention (DLP) solutions can help mitigate the risks associated with accidental insider errors.

Addressing both malicious and accidental insider threats requires a multi-pronged approach, encompassing technical safeguards, user education, and a security-conscious organizational culture.

3.6.4 Data Security Challenges

Data is a valuable asset, often referred to as the "crown jewels" of an organization. Ensuring its confidentiality, integrity, and availability is paramount. This section explores two critical aspects of data security:

- **Data at Rest Encryption:** Data at rest encryption is analogous to storing valuables in a secure safe. It scrambles data when stored on a device, rendering it unreadable in the event of a breach, similar to how a safe protects valuables from theft (Manthiramoorthy et al., 2024). Implementing robust data encryption algorithms is essential for safeguarding data at rest.
- **Data in Transit Encryption:** Data in transit encryption resembles transporting valuables in an armored truck. It protects data while it is being transferred between systems, similar to how an armored truck protects valuables during transportation (Nandakumar et al., 2021). Utilizing secure communication protocols like HTTPS ensures the confidentiality and integrity of data in transit.

By employing a combination of data at rest and data in transit encryption, organizations can significantly reduce the risk of data breaches and unauthorized access to sensitive information.

3.6.5 Hosting Environment/Cloud Security Concerns

Cloud computing offers scalability and flexibility, but security considerations require careful attention. Here, we delve into two key aspects of cloud security:

- **Shared Responsibility Model:** The cloud security landscape operates under a shared responsibility model. This model outlines the division of security responsibilities between the cloud provider and the cloud customer (Gao et al., 2022). Understanding this model and clearly defining the responsibilities of each party is crucial for maintaining a secure cloud environment.
- **Data Residency/Sovereignty:** Data residency refers to the geographical location where data is stored. Data sovereignty regulations may dictate specific requirements for data residency, impacting where an organization can store its data based on legal and compliance considerations (Bian, 2023). Organizations must be aware of relevant data residency regulations and select cloud providers that comply with their specific requirements.

By acknowledging the shared responsibility model and addressing data residency concerns, organizations can leverage the benefits of cloud computing while maintaining a secure environment for their data.

3.7 Multi-Layered Security Constraints

For any common software system, we have to consider multi-layer security constraints that serve as the building blocks for a robust security posture for system applications.

3.7.1 Authentication and Authorization

- **Robust Authentication:** System applications must enforce strong authentication mechanisms to prevent unauthorized access. Multi-factor authentication (MFA) or complex password policies that mandate regular password changes and minimum character length requirements are crucial. These measures significantly increase the difficulty for attackers to gain access with stolen credentials.
- **Principle of Least Privilege:** The principle of least privilege dictates that applications should operate with the minimum set of permissions necessary to fulfill their designated tasks. This minimizes the potential damage caused by compromised credentials or vulnerabilities, as attackers gain access with limited privileges.
- **Granular Authorization:** Fine-grained authorization controls ensure that users can only access the resources and functionalities they are explicitly authorized for. This can be achieved through role-based access control (RBAC) or attribute-based access control (ABAC) mechanisms.

3.7.2 Data Security

- **Encryption at Rest and in Transit:** Sensitive data, both in storage (at rest) and during transmission (in transit), should be encrypted using industry-standard algorithms. This safeguards data confidentiality even if attackers gain access to the system or intercept communication channels.
- **Rigorous Input Validation:** System applications must implement thorough input validation to thwart attacks like SQL injection or cross-site scripting (XSS). Validating all user input ensures that only legitimate data is processed by the application, preventing malicious code injection attempts.
- **Output Encoding:** Applications should encode all output displayed to users to prevent XSS attacks. This ensures that untrusted data is presented securely, mitigating the risk of malicious script execution within the user's browser.

3.7.3 Secure Coding Practices

- **Standardized Secure Coding:** Developers should adhere to established secure coding practices and guidelines throughout the development lifecycle. This minimizes the introduction of vulnerabilities into the application code.
- **Regular Code Reviews:** Implementing a code review process involving security experts or experienced developers can significantly improve code quality. These reviews help identify and address potential security vulnerabilities before the application is deployed.
- **Proactive Vulnerability Scanning:** Regularly scanning system applications for known vulnerabilities using security scanners is essential. This proactive approach allows for prompt patching of vulnerabilities before they can be exploited by attackers.

3.7.4 System Hardening

- **Minimized Attack Surface:** Disabling unnecessary functionalities and features reduces the potential attack surface of the application. This minimizes the number of entry points that attackers can exploit.
- **Timely Updates:** System applications and underlying operating systems should be kept up-to-date with the latest security patches. Patching promptly addresses newly discovered vulnerabilities, making it more difficult for attackers to leverage them.

- **Secure Network Configuration:** Network configurations should be meticulously designed to restrict access to the system application only from authorized sources. Firewalls, intrusion detection/prevention systems (IDS/IPS), and network segmentation can be employed to create a secure network environment.

3.8 Security Constraints for IIS Development

A secure development lifecycle (SDLC) necessitates a multifaceted approach to security constraints. This includes enforcing environment segregation policies to isolate development, testing, and production environments. Precedence for environment variables should be clearly defined during host configuration to prevent accidental exposure of sensitive data. Dependency injection (DI) containers should be configured with appropriate scope selection to minimize attack surfaces. Rate limiting configurations can throttle suspicious activity and mitigate brute-force attacks. Implementing anti-forgery tokens safeguards against unauthorized actions by validating user requests. Finally, a synergistic approach to authorization that combines role-based access control (RBAC) with attribute-based access control (ABAC) can provide granular control over user permissions based on context and dynamic attributes. By implementing these security constraints throughout the SDLC, institutes can significantly reduce the risk of vulnerabilities and protect sensitive information within their information systems.

3.8.1 Environment Segregation Policy

The software development lifecycle (SDLC) is a structured process for planning, designing, developing, testing, deploying, and maintaining software applications. Within the SDLC, the utilization of distinct environments is a critical element in ensuring the production of high-fidelity software. Each environment serves a specific function, collectively contributing to the overall stability and efficacy of the final product. This paper explores the significance of four prominent environments within the SDLC: Development (Dev), Test, Staging, and Production.

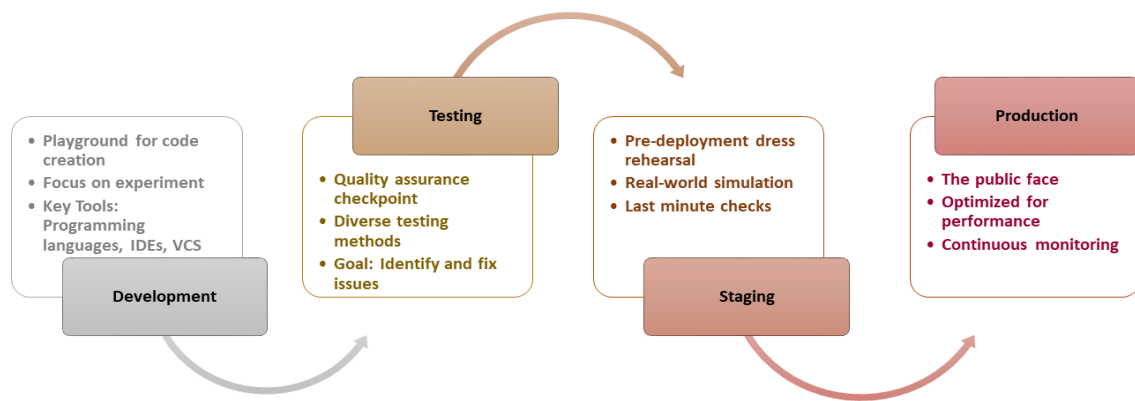


Figure 5 Sequence of Environments for System Development

Development environment

The Dev environment serves as the foundation for software creation, akin to a controlled laboratory for experimentation. In this milieu, developers engage in the core activities of writing, editing, and testing their code. The primary objective is to provide a sandboxed space for iterative development without compromising the integrity of the primary codebase.

The Dev environment empowers developers to experiment with novel features, refactor existing code, and rectify identified errors. It fosters a dynamic atmosphere that encourages creativity and fosters innovative solutions. However, due to the inherently fluid nature of this environment, meticulous version control and frequent code integration practices are essential to prevent conflicts during later stages of development.

Test environment

As software progresses through the SDLC pipeline, the Test environment assumes a pivotal role in meticulously scrutinizing the application before it is made available to end-users. This environment is meticulously configured to emulate real-world conditions, enabling testers to conduct a comprehensive battery of tests designed to detect and eliminate bugs or glitches that may have remained undiscovered during the Dev phase.

Test environments provide quality assurance (QA) teams with the platform to execute various testing methodologies, encompassing unit testing, integration testing, and user acceptance testing (UAT). Through this rigorous testing process, they guarantee that the application adheres to established quality benchmarks and functions as intended. Any

issues identified during testing are communicated back to the developers for rectification before the software progresses to the subsequent phase.

Staging environment

The Staging environment can be likened to a dress rehearsal preceding a theatrical performance. In this environment, the software is meticulously staged in a meticulously configured environment that closely mirrors the Production setup, encompassing server configurations, databases, and network settings. Staging serves as the final checkpoint before deploying the application to the intended user base.

Within the Staging environment, development teams have the opportunity to simulate real-world usage scenarios to pinpoint potential performance bottlenecks, compatibility issues, and any lingering glitches. Given the close resemblance between Staging and Production environments, this phase offers a more accurate assessment of the application's behavior upon deployment.

Production environment

The Production environment represents the culmination of the software development journey. This is where the application is finally unveiled to the public or the designated user base. The Production environment is meticulously optimized for performance, reliability, and security, as any issues encountered at this stage can have a direct impact on users and business operations.

Maintaining the Production environment necessitates continuous monitoring, regular updates, and the implementation of a swift response mechanism to address any unanticipated issues that may surface. Stability is paramount within the Production environment, as users rely on the application's functionality to fulfill their needs.

3.8.2 Precedence of Environment Variables

Configuration management

Within the realm of modern application development, effective configuration management across various development stages is paramount. This practice ensures distinct environments (Development, Test, Staging, Production) operate with tailored settings, fostering a robust development lifecycle. For examine environment, we used .NET Core application configuration consideration. Here, we'll explore a configuration paradigm utilizing separate JSON files for each environment.

Directory structure:

- appsettings.json (Base Configuration): This file stores configuration values that are common across all environments. For example, application name, logging level, etc.
- appsettings.{Environment}.json (Environment Specific): These files store configuration pertinent to individual environments. Examples encompass database connection strings, API keys, or URLs. Such as:
 - appsettings.Development.json (Dev Specific)
 - appsettings.Test.json (Test Specific)
 - appsettings.Staging.json (Staging Specific)
 - appsettings.Production.json (Production Specific)

In configuration settings, there should never store sensitive information such as connection strings or API keys directly in this file, especially within an off-host context. In real-world scenarios, consider employing environment variables or a secure secrets management system for production environments.

This host configuration of the application retrieves the current environment variable named `ASPNETCORE_ENVIRONMENT` (typically set within the launch settings or deployment configuration) and subsequently loads the corresponding environment-specific JSON file alongside the base `appsettings.json`.

This methodology separates base configuration from environment-specific settings, promoting maintainability and secure handling of sensitive information in production environments (employing environment variables or a secrets management system). By adopting this approach, you can effectively manage configuration throughout the development lifecycle within an application configuration setting.

Within the .NET Core hosting ecosystem, environment variables play a crucial role in configuring application behavior. However, the order in which these variables are loaded and potentially overridden determines the final configuration used by the application. This paper explores the precedence of environment variables in .NET Core hosting, ensuring a well-defined and predictable configuration management approach.

Hierarchy of configuration sources

.NET Core adheres to a well-defined order when loading configuration data, with some sources taking precedence over others. Here's a breakdown of this hierarchy, starting from the highest precedence:

Host Configuration (ASPNETCORE_ prefixed variables): These environment variables are designated specifically for the .NET Core host process. They are typically used to configure core aspects like the application environment itself (e.g., `ASPNETCORE_ENVIRONMENT=Development`). Notably, these variables are accessed through the `IConfiguration` interface within application code and are set during host creation using methods like: `WebHost.CreateDefaultBuilder().ConfigureAppConfiguration(...)`.

Environment-Specific Configuration Files (`appsettings.{Environment}.json`): This JSON file format provides a mechanism to define configuration settings tailored to specific environments. The placeholder `{Environment}` is dynamically replaced with the actual value set for the `ASPNETCORE_ENVIRONMENT` variable (e.g., `appsettings.Development.json`). These environment-specific settings are readily accessible using the `IConfiguration` interface within the application code ([Microsoft],[n.d.]).

Default Configuration File (`appsettings.json`): This JSON file serves as the foundation for configuration settings, acting as a default for all environments. If a specific setting is absent from the environment-specific file (`appsettings.{Environment}.json`), the value defined in `appsettings.json` will be used. Similar to other configuration sources, these settings are accessed through the `IConfiguration` interface within the application code ([Microsoft],[n.d.]).

User Secrets: User secrets offer a secure approach to storing sensitive configuration values outside of the application's source code. These secrets are typically accessed using libraries specific to the chosen implementation, such as `Microsoft.Extensions.Configuration.AzureKeyVault` for integration with Azure Key Vault.

Operating System specified Environment Variables (without `ASPNETCORE_` prefix): These are standard environment variables set on the operating system or through

container configurations. They are accessible using the IConfiguration interface within the application code.

Command-Line Arguments: Arguments passed during application startup through the command line can be used to override any previously loaded configuration values. These arguments are typically accessed using the IArguments interface within the application's Main method or through libraries like CommandLineParser.

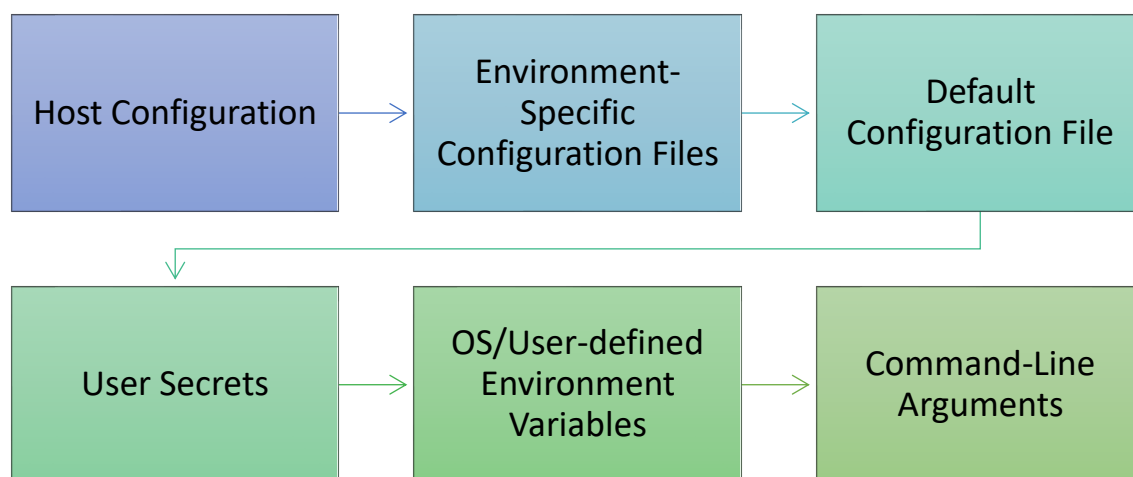


Figure 6 Precedence of Environment Variables Hierarchy

Key considerations for effective configuration management

- **Prioritize Dedicated Host Configuration:** It is recommended to avoid using environment variables prefixed with ASPNETCORE_ for application configuration purposes. This prefix is specifically reserved for host-related settings.
- **Leverage Environment-Specific Configuration:** Utilize environment-specific configuration files (appsettings.{Environment}.json) to define settings tailored to different deployment environments.
- **Embrace User Secrets for Sensitive Data:** For sensitive configuration data, user secrets provide a secure storage mechanism, enhancing overall application security.
- **Reserve Command-Line Arguments for Dynamic Overrides:** Command-line arguments are best suited for occasional overrides during application execution, offering flexibility for specific use cases.

By adhering to this established precedence and best practices, developers can ensure a well-defined and predictable configuration management approach within their .NET Core applications. This, in turn, fosters a more robust and maintainable codebase.

3.8.3 Scope selection for DI Container

Dependency injection (DI) is a prevalent design pattern in modern software development. It promotes loose coupling and facilitates unit testing by managing the object lifecycle of dependencies. Selecting the appropriate scope for injected dependencies within software hosting environments significantly impacts application behavior and resource management.

DI containers manage the creation and lifecycle of objects (dependencies) required by a class, fostering loose coupling and enhancing testability. However, a critical aspect of DI involves selecting the appropriate scope for each injected dependency. This selection significantly influences application behavior and resource management, particularly within software hosting environments.

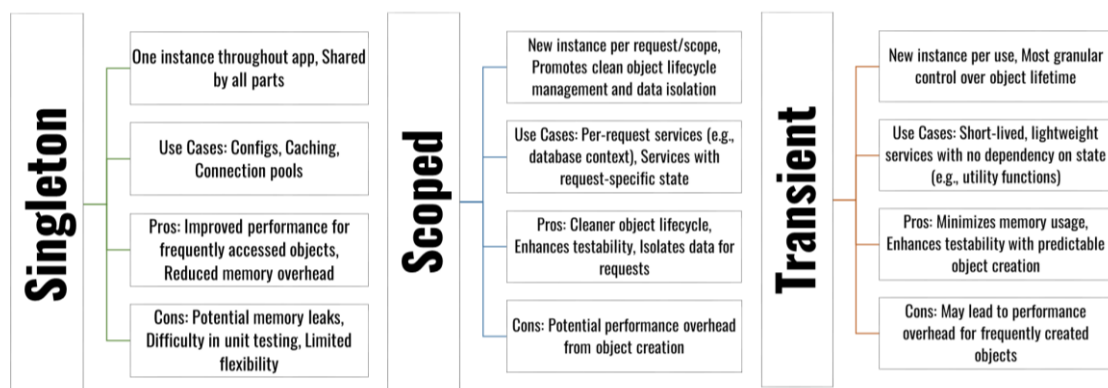


Figure 7 Different Scopes for Dependency Injection Instances

Dependency injection (DI) containers offer various scopes to manage the lifecycle of injected dependencies. These scopes determine how many instances of a dependency are created and for how long they persist within the application. Selecting the appropriate scope is crucial for optimizing application performance, resource utilization, and maintainability in software hosting environments.

Singleton Scope

The DI container creates a single instance of the dependency for the entire application's lifetime. This instance is shared by all parts of the application that request it. Singletons are well-suited for application-wide configurations, caching services (with proper invalidation mechanisms to ensure data freshness), and connection pools (with

appropriate disposal mechanisms to avoid resource exhaustion). A key benefit of singletons is improved performance for frequently accessed, thread-safe objects, as there is no overhead associated with creating multiple instances. Additionally, singletons can reduce memory overhead by minimizing object creation. However, there are also drawbacks to consider. Singletons can potentially lead to memory leaks if not properly managed, especially if the singleton holds onto resources that are not disposed of when no longer needed. Unit testing singleton dependencies can also be challenging due to their shared state. Finally, singletons offer limited flexibility for scenarios requiring isolated behavior, as all parts of the application share the same instance.

Scoped Scope

Scoped scope is particularly advantageous in web applications, where each HTTP request represents a distinct unit of work. By creating a new instance of the scoped dependency for each request, we ensure clean separation of concerns and prevent data contamination between requests. This is especially valuable when working with database contexts or other services that manage request-specific state. For instance, a web application might utilize a scoped database context to isolate database operations for each user session. This approach guarantees that changes made by one user do not inadvertently affect the data accessed by another user. Scoped dependencies also promote better testability. Unit tests can be designed to interact with a predictable instance of the dependency, simplifying test setup and reducing the risk of unintended side effects.

Transient Scope

Transient scope offers the most fine-grained control over the lifetime of injected dependencies. The DI container creates a new instance of the transient dependency every time it is requested. This approach is ideal for short-lived, lightweight services with no dependency on state. Examples include utility functions performing simple calculations or loggers that write to temporary files. Transient scope minimizes memory usage by ensuring that objects are only created and retained for the duration of their immediate use. This is particularly beneficial for cloud-based deployments, where resource optimization is paramount. Additionally, transient scope promotes loose coupling and simplifies unit testing, as tests can interact with a new instance of the dependency for each test case, eliminating potential side effects from previous tests.

Selecting the appropriate DI scope hinges on the specific needs of the software component and the software hosting environment. In software hosting, singletons can be beneficial

for shared configurations or caching, while scoped instances are well-suited for managing request-specific data in web applications. Transient instances offer granular control and minimize memory usage for short-lived objects. Understanding the characteristics and use cases of each scope empowers developers to make informed decisions that contribute to robust, efficient, and maintainable software hosting solutions.

3.8.4 Rate Limit Configuration

Rate limiting constraints are a fundamental security and performance optimization technique in various systems. They regulate the number of requests or actions a user or entity can perform within a specific time frame. While a singular, universal mathematical equation cannot represent all rate limiting scenarios, several formulations effectively model different rate limiting behaviors. Here we have explored these mathematical frameworks for analyzing and comparing rate limiting constraints.

Fixed rate limiting

The simplest form of rate limiting enforces a fixed maximum number of requests (***R***) allowed per unit of time (***T***). This can be mathematically expressed as:

$$requests(t) \leq R \quad \dots\dots\dots (4.1)$$

Here, $requests(t)$ represents the total number of requests received within the time interval ***t***. This approach offers a clear and predictable constraint, suitable for scenarios where a consistent request rate is desired.

Leaky bucket model

The leaky bucket model provides more flexibility by allowing for bursts of requests exceeding the average rate. It functions like a leaky reservoir. Here, the bucket has a limited capacity (***C***), representing the maximum number of queued requests. A constant leak rate (***r***) determines the number of requests allowed out of the bucket per unit time. This model can be mathematically represented as:

$$requests(t) \leq C + r * (t - t_{lr}) \quad \dots\dots\dots (4.2)$$

where:

- ***C*** denotes the bucket capacity.
- ***r*** signifies the leak rate.
- ***t_{lr}*** represents the timestamp of the most recent request.

This formulation allows for a burst of requests exceeding the average rate (**r**) as long as the bucket has enough capacity (**C**). However, if requests arrive faster than the leak rate can accommodate, they will be queued and eventually dropped. This model is well-suited for scenarios where occasional bursts of activity are expected.

The token bucket model

Similar to the leaky bucket, the token bucket model manages request bursts but uses a different approach. Instead of a leaky reservoir, tokens are generated at a constant rate (**r**). A request requires one token, and the bucket has a limited capacity (**C**) to hold tokens. This allows for bursts within the capacity limit. Mathematically, this can be represented as:

$$tokens(t) \geq 1 \quad \dots\dots\dots (4.3.1)$$

#At least one token is needed for a request;

$$tokens(t) = \min(C, tokens(t_{lr}) + r * (t - t_{lr})) \quad \dots\dots\dots (4.3.2)$$

$$requests(t) \leq tokens(t) \quad \dots\dots\dots (4.3.1)$$

#A request consumes one token;

where:

- $tokens(t)$ represents the number of tokens available at time **t**.

With this model, requests are allowed as long as there are sufficient tokens available ($tokens(t) \geq 1$). Tokens are replenished at a constant rate (**r**). This approach enables controlled bursts of activity up to the bucket capacity (**C**).

The sliding window rate limiting

The sliding window approach focuses on the total number of requests within a specific time window (**W**). It can be mathematically expressed as:

$$\sum requests(t_i) \text{ for } t_i \text{ in window } W \leq R \quad \dots\dots\dots (4.4)$$

Here, the summation iterates over all requests ($requests(t_i)$) received within the defined window **W**. This approach ensures that the overall request rate remains within the specified limit (**R**) over the time window. It's suitable for scenarios where sustained high request volumes are a concern.

Dynamic rate limiting adaptations

Current practices often rely on static rate thresholds. Promising research directions lie in exploring dynamic approaches that adjust these limits in real-time. Constraint programming approaches could be harnessed to analyze user behavior, threat intelligence feeds, and system load. By identifying anomalous activity patterns, these algorithms could dynamically adjust rate limits, offering a more responsive defense against evolving threats.

Traditional rate limiting applies a single limit to a specific resource. Our model delve into more granular approaches, setting differentiated rate limits based on user roles, request types, or even the specific data being accessed. This would enable a more nuanced control over access and resource allocation, catering to the varying risk profiles of different users and requests.

Furthermore, incorporating additional context beyond just request volume holds significant promise. By applying our model, the constraints could incorporate factors like geographical location, time of day, or user history to create context-aware rate limits. This would enhance the ability to differentiate legitimate users from malicious actors attempting exploits.

3.8.5 Anti-forgery Implementation

Web applications have become an indispensable facet of modern life, managing sensitive data and facilitating critical transactions. However, this ubiquity also renders them susceptible to various security threats. One such threat is Cross-Site Request Forgery (CSRF), a malicious technique where an attacker exploits a user's authenticated session to perform unauthorized actions on a web application. Anti-Forgery Validation (AFV) emerges as a critical security measure that safeguards web applications against CSRF attacks. This paper delves into the significance of AFV for robust web application security.

Cross-site request forgery (CSRF)

CSRF attacks capitalize on a user's existing, trusted session with a web application. The attacker tricks the user's browser into transmitting a forged HTTP request to the application. Since the user's browser already possesses valid authentication credentials (e.g., cookies), the application often processes the request without recognizing it as illegitimate. This can lead to a range of consequences, such as unauthorized money transfers, sensitive data modification, or account profile changes.

AFV serves as a potent defense mechanism against CSRF attacks. It operates by incorporating a hidden form element, known as an anti-forgery token, within web application forms. This token is a unique value generated by the server for each user session. When the user submits the form, the token is included in the request alongside any other form data. The server validates the submitted token against the one it maintains for the user's session. If the tokens match, the request is deemed legitimate; otherwise, it's rejected.

Implementing AFV in web applications offers several security advantages:

- **Enhanced CSRF Protection:** AFV significantly reduces the risk of successful CSRF attacks by validating the authenticity of form submissions.
- **Improved User Trust:** By safeguarding against unauthorized actions, AFV fosters user confidence in the application's security posture.
- **Reduced Attack Surface:** AFV narrows the window of vulnerability for CSRF attacks, making web applications less susceptible to exploitation.
- **Minimal Performance Impact:** The computational overhead associated with AFV is generally negligible, ensuring minimal performance degradation.

While AFV offers significant security benefits, proper implementation is crucial for its effectiveness. Here are some key considerations:

- **Token Generation:** Tokens should be cryptographically secure, generated using a strong random number generator, and refreshed periodically to prevent predictability.
- **Token Storage:** Tokens should be stored securely on the server-side and not be exposed to the client.
- **Token Validation:** Server-side validation of the submitted token against the session-specific token is essential to ensure request legitimacy.
- **Framework Integration:** Many web development frameworks provide built-in AFV functionalities that simplify implementation.

Anti-Forgery Validation is an indispensable security measure for web applications. By mitigating the risks associated with CSRF attacks, AFV protects sensitive user data and

fosters trust in the application. Understanding the mechanics of CSRF attacks and the benefits of AFV empowers developers to implement robust security practices and create a more secure web environment for users.

Anti-forgery validation, while a valuable tool, has some limitations to consider:

- **Client-Side Reliance:** It relies on tokens generated on the client-side (user's browser). A sophisticated attacker could potentially bypass this by crafting malicious scripts to steal or forge the token.
- **CSRF (Cross-Site Request Forgery) Focus:** It primarily protects against CSRF attacks, where an attacker tricks a user into unknowingly performing unauthorized actions on a trusted website. It's not effective against other attack vectors like SQL injection or brute-force attacks.
- **Custom Applications:** Anti-forgery validation often works through forms and browser interactions. For custom applications or API requests that don't use forms, implementing the validation can be more complex.
- **Potential for User Experience Friction:** Including the anti-forgery token as a hidden form field can slightly increase the payload size of web pages, potentially impacting load times.

3.8.6 Synergistic Approach for Authorization

The cornerstone of secure systems lies in robust authorization mechanisms that control access to resources and functionalities. Traditional Role-Based Access Control (RBAC) has served as a foundational approach, but the evolving landscape of applications and microservices demands a more granular and adaptable solution. We have discussed authorization procedures as complementary techniques that empower fine-grained access control within modern systems.

Claim-based authorization

Claim-Based Authorization (CBA) departs from the centralized control of RBAC. Instead, it leverages claims – assertions about an identity or resource – to determine access permissions. These claims can originate from various sources, including user profiles, security tokens, or external identity providers. This decentralized approach offers several advantages:

- **Enhanced Flexibility:** Claims can be tailored to specific attributes or contextual information, enabling more granular access control decisions. For instance, a claim might indicate a user's location, device type, or specific permissions granted within an application.
- **Improved Reusability:** Claims are independent of specific resources, allowing them to be reused across different applications or services. This reduces redundancy and simplifies authorization logic.
- **Simplified Identity Management:** By decoupling access control from user roles, CBA facilitates the integration of diverse identity providers and simplifies user management.

However, CBA also presents some challenges:

- **Increased Complexity:** Managing a distributed network of claims can be more intricate compared to RBAC's centralized model.
- **Potential Performance Overhead:** Evaluating complex claims-based conditions might introduce slight performance overheads compared to simpler RBAC checks.

The realm of secure systems hinges on well-defined authorization mechanisms that govern access to resources and functionalities. Role-Based Access Control (RBAC) has long been a mainstay in this domain, offering a structured approach to access management. However, the ever-evolving landscape of applications and microservices demands a more flexible and granular control over access decisions. This paper explores the relationship between RBAC and Claim-Based Authorization (CBA), demonstrating how RBAC can be viewed as a specialized form of CBA.

Role-based access control

While CBA offers a broad foundation, Role-Based Access Control (RBAC) leverages claims in a more structured manner. RBAC defines three core entities:

- **Roles:** Predefined sets of permissions associated with specific functionalities within a system.
- **Users:** System actors assigned one or more roles.
- **Permissions:** Granular access rights associated with specific resources or actions.

RBAC enforces access control based on the relationship between users and their assigned roles, which in turn determine the permissions they possess. This approach offers several advantages:

- **Simplicity:** RBAC provides a clear and intuitive model for access control, making it easy to understand and manage.
- **Efficiency:** Evaluating access based on pre-defined roles streamlines authorization decisions.
- **Reduced Complexity:** RBAC simplifies claim management by focusing on a well-defined set of roles.

However, RBAC also has limitations:

- **Limited Flexibility:** The pre-defined nature of roles might not cater to all access control scenarios requiring dynamic conditions.
- **Scalability Challenges:** Managing a large number of roles and their associated permissions can become cumbersome in complex systems.

RBAC as a specialized form of CBA

By delving deeper, we can recognize RBAC as a specialized form of CBA. Here's how:

- **Roles as Claims:** In essence, roles in RBAC represent a specific type of claim about a user's identity. A user being assigned a role translates to a claim stating their membership in that role.
- **Permissions as Claims:** Similarly, permissions associated with roles can be viewed as claims. Owning a role implies possessing the associated permissions as claims.
- **Policy Enforcement:** RBAC enforces access control policies based on user-role mappings, which can be seen as a specific way of evaluating claims. These policies determine whether a user's role-based claims grant them access to a resource.

Therefore, RBAC leverages the core principles of CBA by utilizing claims (roles and permissions) to control access. However, it restricts the types of claims considered and simplifies the evaluation process by focusing on pre-defined roles.

Claim-Based Authorization offers a versatile foundation for modern authorization systems. Role-Based Access Control, by leveraging claims in a structured manner, provides a clear and efficient approach for access control in many scenarios. Understanding their relationship empowers developers to choose the most appropriate technique for their specific needs. While RBAC offers simplicity for well-defined roles, CBA's flexibility caters to more intricate access control requirements in modern distributed systems.

Table 1 Comparison between Authorization Domains

Feature	Claim-Based (CBA)	Role-Based (RBAC)	Policy-Based (PBA)
Basis for Access Control	Claims (attributes about identities or resources)	Predefined Roles	Policies (rules defining access criteria)
Flexibility	Highly flexible, supports diverse attributes	Limited, focuses on predefined roles	Flexible, allows complex access rules
Reusability	Claims can be reused across different resources	Roles are resource-specific	Policies can be reused across different resources
Complexity	More complex to manage due to distributed claims	Simpler to understand and manage	Can be complex to define and manage intricate policies
Scalability	Scales well with diverse access scenarios	May struggle with a large number of roles	Scales well with complex access requirements
Example Claims	User role, location, device type, specific permission	Administrator, Editor, Reader	User role = "Admin" AND resource type = "Finance"

Policy-Based Authorization

Policy-Based Authorization (PBA) introduces a rule-based approach to authorization. Policies define the criteria for granting or denying access to resources. These policies can

be expressed in various languages, allowing for the specification of complex access control logic. PBA offers several benefits:

- **Expressive Power:** Policies can incorporate a wide range of conditions, including claims, user roles, environmental factors, or resource attributes. This enables the creation of highly nuanced access control rules.
- **Clear Separation of Concerns:** Policies separate authorization logic from application code, promoting maintainability and code reuse.
- **Centralized Management:** Policies can be centrally managed and enforced across different parts of the system, ensuring consistency and simplifying authorization governance.

However, PBA also has limitations:

- **Increased Development Effort:** Defining and managing complex policies can be more time-consuming compared to simpler RBAC models.
- **Potential for Rule Conflicts:** With intricate policy sets, there's a risk of conflicting rules, requiring careful design and testing to ensure consistent authorization behavior.

A Synergistic Approach

While CBA and PBA offer distinct advantages, their true power lies in their synergy. Modern authorization systems often combine these approaches to create a robust and adaptable framework. Claims provide the foundation for expressing authorization requirements, while policies define the rules for evaluating these claims and making access decisions. This combined approach offers several advantages:

- **Granular Control:** Claims and policies together enable highly granular access control, catering to the diverse needs of modern applications.
- **Flexibility and Reusability:** The combination fosters flexibility by leveraging claims for varied access control scenarios and facilitates policy reuse across different services.
- **Maintainability and Governance:** Separating claims from policies promotes maintainability, while centralized policy management ensures consistent authorization enforcement.

In conclusion, Claim-Based and Policy-Based Authorization represent powerful and complementary approaches to authorization in modern systems. By understanding their individual strengths and limitations, developers can leverage them synergistically to create robust and adaptable authorization frameworks, safeguarding access to critical resources and functionalities within complex software ecosystems.

3.9 The Secured Institutional Information System Development Model (SIISDM)

This section presents the culmination of our efforts to integrate the robust security framework of ISO/IEC 27001:2022(E) (International Organization for Standardization (ISO), 2022) with the established practices of Microsoft's Security Development Lifecycle (SDL). Through this integration, we have developed the Institutional Security Assurance Model (SIISDM), a comprehensive model designed to elevate the security posture of information systems within individual institution.

The core of SIISDM lies in its seven distinct components. Five of these components represent the core phases of the model, mirroring the traditional SDLC phases: requirements, design, implementation, verification, and release. These core phases are structured with mandatory checks and approval processes. This ensures that security and privacy best practices are systematically addressed throughout the development lifecycle of information systems.

Flanking these core phases are two additional, yet crucial, supporting security activities: training and response. The training component emphasizes the importance of equipping personnel with the necessary knowledge and skills to effectively implement the SIISDM.

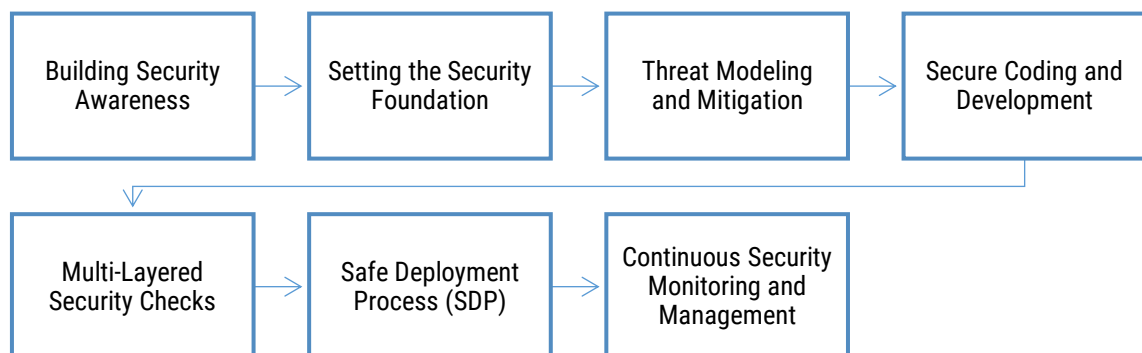


Figure 8 The Secured Institutional Information System Development Model (SIISDM)

3.9.1 Building Security Awareness

All Institute stakeholders undergo general security and privacy training, with role-specific training offered as well. Initial training occurs upon hire, followed by annual refreshers. Developers and engineers receive further training on secure development principles and current security trends. Additionally, Institute encourages and facilitates access to advanced security and privacy training for all employees, interns, contractors, and third parties.

3.9.2 Setting the Security Foundation

Security and privacy requirements are the cornerstone of secure software development, defined for each product, service, and feature. Development teams consider factors like data handling, known threats, regulations, and lessons learned from past incidents. These requirements are documented and continuously reviewed to reflect changes in functionality and the threat landscape.

3.9.3 Threat Modeling and Mitigation

Once requirements are defined, software design commences. Threat modeling, a vital component, helps identify, categorize, and prioritize potential threats based on risk. As the software evolves, threat models are continuously updated to reflect changes.

The process begins by defining software components and their interactions. Data Flow Diagrams (DFDs) visually depict data flow, data types, ports, and protocols. DFDs help identify and prioritize threats for mitigation, which are then incorporated into the security requirements.

Institute's Threat Modeling Tool empowers service teams to communicate security design, analyze designs for vulnerabilities using established methodologies and recommend and manage mitigation strategies. Before release, all threat models undergo a thorough review for accuracy and completeness, ensuring appropriate mitigation for unacceptable risks.

3.9.4 Secure Coding and Development

Implementation involves developers writing code according to the established plan. Institute equips developers with a suite of secure development tools that streamline the implementation of security, privacy, and functional requirements. These tools include compilers, secure development environments, and built-in security checks.

3.9.5 Multi-Layered Security Checks

Prior to release, code undergoes rigorous checks and approvals to ensure adherence to SDL, design requirements, and coding best practices. Manual code reviews are conducted by individuals independent of the developer, minimizing the risk of errors or vulnerabilities. Additionally, automated checks analyze code during check-in and compilation stages. These checks fall into several categories:

- **Static Code Analysis:** Analyzes source code for potential flaws like embedded credentials.
- **Binary Analysis:** Assesses vulnerabilities at the binary code level for production readiness.
- **Credential and Secret Scanner:** Identifies potential exposure of credentials and secrets in code and configuration files.
- **Encryption Scanning:** Validates encryption best practices in code and execution.
- **Fuzz Testing:** Uses malformed data to exercise APIs and parsers, uncovering vulnerabilities and validating error handling.
- **Configuration Validation:** Analyzes production system configurations against security standards and best practices.
- **Component Governance (CG):** Detects open-source software and verifies version, vulnerability information, and legal obligations.

If issues arise during the review process, the developer is notified and must rectify them before resubmission. Additionally, penetration testing, conducted by both internal and external teams, is used to detect security flaws potentially missed by other methods.

3.9.6 Safe Deployment Process (SDP)

Successful completion of security tests and reviews doesn't lead to immediate global release. Instead, builds are systematically rolled out to progressively larger groups through a Safe Deployment Process (SDP). These rings typically include:

- Phase 0: The development team responsible for the service or feature.
- Phase 1: All Institute stakeholders.
- Phase 2: Users outside of Institute who have privileged for the targeted release channel.

- Phase 3: Global standard release in sub-phases.

Builds remain in each ring for a designated period, including high-load testing phases (except Phase 3, which has already undergone extensive testing in previous rings).

3.9.7 Continuous Security Monitoring and Management

Released services are extensively monitored and logged after deployment using a centralized, near-real-time monitoring system to identify potential security incidents.

By integrating security considerations throughout the development lifecycle, the SIISDM fosters a culture of security awareness and empowers teams to build secure and reliable software solutions.

The response component focuses on establishing a robust incident response strategy to effectively address potential security breaches and vulnerabilities after deployment.

3.10 Implementation and Testing of SIISDM

The implementation of the model goes on hosted system applications at Live Stand-alone Production Server of PSTU IIS.

Table 2 SIISDM Applied System Applications

System project name	Front-end platform	Back-end platform	Link to the Live System
Student Skills Training Portal EDGE_CSIT PSTU, 2024-Current	C# ASP.NET Core (MVC)	C# .NET 8, Entity Framework, SQL Server, IIS	https://edge.csit.pstu.ac.bd/
PSTU Undergraduate Admission System, 2020-Current	C# ASP.NET Core (MVC)	C# .NET 6, Entity Framework, SQL Server, IIS	https://admission.pstu.ac.bd
PSTU Diary Back-End Deployment, 2023-Current	Android (kotlin)	Python (Django REST framework) & MySQL	https://dairy.pstu.ac.bd/
PSTU Student Identity and Card Management System, 2023-Current	C# ASP.NET Core (MVC)	C# .NET 6, Entity Framework, SQL Server, IIS	https://piis.pstu.ac.bd:4040/
PSTU Institute Information System (PIIS), 2020-Current	C# ASP.NET Core (MVC)	C# .NET Core 2.2, Entity Framework, SQL Server, IIS	https://piis.pstu.ac.bd/

Considering the scenario for a software deployment where the host configuration addressed to a development level environment variable which is unreachable from the production environment. Hence the database instance has configured with scoped services by the dependency injection (DI) that maintain the instance for every client user. On production live application, the parallel multiple request invoked at same time to the request processing pipeline. The unreachable environment variable finding has been continuing through the middleware until it gets any attribute or cancelled by invoked instance. That stacked the instance on runtime memory and the performance decreased geometrically and the decrease rate is proportional to the number of client users.

To mitigate this system vulnerability, we apply the SIISDM and ensured that nothing other than right production configuration and/or environment variables has addressed during the development lifecycle.

On the other hand, the rate limit configuration should have to apply dynamically over the security constraints during code implementation based on user roles, request types, or even the specific data being accessed. Hence the service configuration is used to add the rate limit policies for both client and IP address through the dependency injection (DI) as singleton scope by default. As a result, it is statically configured so that, it is difficult to extend within security constraints dynamically.

To mitigate this issue, we extend the dependency injection of rate limit configuration for scoped scope so that the client based instances are become maintainable in memory or even with the distributed system.

CHAPTER IV

RESULT AND DISCUSSION

4.1 Introduction

The following sections will delve deeper into the specifics of each SIISDM component, exploring how they interact to create a comprehensive security assurance model for our institution's information systems. We will then discuss the potential benefits and limitations of the SIISDM, offering insights into its practical application and areas for further refinement.

4.2 Security Threats and Issues

According to the survey response and selected live server security logs, the security threads for developing institutional information system and their potential measures have enlisted (Appendix A).

Table 3 Security Issues for Developing IIS

Category	Security Issue	Risk Level
Data Security	Data Breach	High
	Data Loss	High
	Data Tampering	Medium
	Unencrypted Data	Medium
Access Control	Unauthorized Access	High
	Least Privilege Not Enforced	Medium
	Account Takeover	Medium
Network Security	Denial-of-Service (DoS) Attack	High
	Malware Infection	High
	Man-in-the-Middle Attack	Medium
System Vulnerabilities	Unpatched Software	High
	Misconfiguration	Medium
	Zero-Day Exploit	High
Misconfiguration	Security Settings Not Enabled	Medium
	Default Passwords Not Changed	Medium
Insider Threats	Disgruntled Employee	High
	Accidental Insider Errors	Medium
Social Engineering	Phishing	Medium
	Pretexting	Medium
Physical Security	Uncontrolled Physical Access	Medium
	Theft of Physical Devices	Medium

At a glance, the total identified 21 issues categorized in 8 broad categories based on the robust security framework of ISO/IEC 27001:2022(E) where 8 have marked as risk level of high and other 13 have marked as risk level of medium.

The survey identified several common challenges faced by institutions in their efforts to implement and maintain secure information systems. A significant portion of institutions expressed a need for more comprehensive guidance, awareness, and training programs to equip staff with the necessary knowledge and skills to address security threats. Many institutions reported inadequate budget or resources allocated to security measures, hindering their ability to implement effective security solutions. Existing security policies may not be adequately implemented or enforced, leading to vulnerabilities and compliance issues. The rapidly evolving landscape of cyber threats poses a challenge for institutions to stay up-to-date with the latest security best practices and technologies. Moreover, the lack of in-house expertise in cybersecurity can hinder institutions' ability to effectively assess and mitigate security risks. To mitigate these risks, institutions primarily rely on firewalls, antivirus/antimalware software, and intrusion detection/prevention systems instead of practice of focusing for framework of the development of the secure institutional information system.

Despite these challenges, the survey also revealed some positive trends. A majority of institutions (87.5%) have interested to practice any of established formal guidelines to ensure system security, indicating a commitment to addressing security concerns. A significant portion of institutions (25%) rated their awareness of software security threats as high or very high, suggesting a growing recognition of the importance of information security.

4.3 Security Assurance Model for IIS

The Secured Institutional Information System Development Model (SIISDM) represents a novel approach to enhancing information system security by combining the rigorous standards of ISO/IEC 27001:2022 with the proven methodologies of Microsoft's Security Development Lifecycle (SDL). This integration results in a comprehensive framework that addresses security and privacy concerns at every stage of the system development lifecycle.

SIISDM is composed of seven distinct components, five of which align with the traditional SDLC phases: requirements, design, implementation, verification, and release.

By incorporating mandatory checks and approval processes within these phases, SIISDM ensures that security and privacy best practices are systematically integrated into the development process. This rigorous approach helps to mitigate risks and vulnerabilities that may arise during the development and deployment of information systems.

Table 4 Comparison between different Security Assurance Model for IIS

Model	Focus	Key Features
Secure Development Lifecycle (SDL)	Proactive security throughout the SDLC	Threat modeling, secure coding, vulnerability testing, penetration testing
Building Security In (BSI)	Integrated security in SDLC	Secure design, coding practices, verification, validation
OWASP Top 10	Critical web application vulnerabilities	List of top 10 risks
Secured Institutional Information System Development Model (SIISDM)	Comprehensive security framework for IIS	Combines ISO/IEC 27001 and SDL, addresses security and privacy at all SDLC stages

A comparative analysis of the Secure Development Lifecycle (SDL), Building Security In (BSI), OWASP Top 10, and the Secured Institutional Information System Development Model (SIISDM) reveals distinct approaches to securing information systems. While SDL and BSI emphasize proactive security measures throughout the development lifecycle, OWASP Top 10 focuses on addressing specific vulnerabilities. SIISDM, on the other hand, provides a comprehensive framework that integrates elements from both SDL and ISO/IEC 27001, offering a more holistic approach to security focusing security of IIS. Thus ensure the capability of SIISDM for assisting institutes/organizations in determining their readiness to develop secure information system.

4.4 Adaptability of SIISDM

The Secured Institutional Information System Development Model (SIISDM) was successfully implemented on several live applications hosted at PSTU IIS, including the Student Skills Training Portal, PSTU Undergraduate Admission System, PSTU Diary Back-End Deployment, PSTU Student Identity and Card Management System, and PSTU Institute Information System (PIIS). This deployment facilitated the identification and

mitigation of security vulnerabilities that could have compromised system performance and data integrity.

4.4.1 Precedence of Environment Variables

One critical vulnerability identified involved a development-level environment variable being referenced in the production environment. This caused the database to create scoped services using dependency injection (DI) for each user, leading to memory overload and performance degradation as user traffic increased. By adhering to SIISDM principles, the development process ensured that only valid production configurations were used, eliminating this vulnerability.

After applying the SIISDM and ensured that nothing other than right production configuration and/or environment variables has addressed during the development lifecycle, the request process timing has been recorded from client-site endpoint from both outside and inside of the host network (Appendix B and Appendix C).

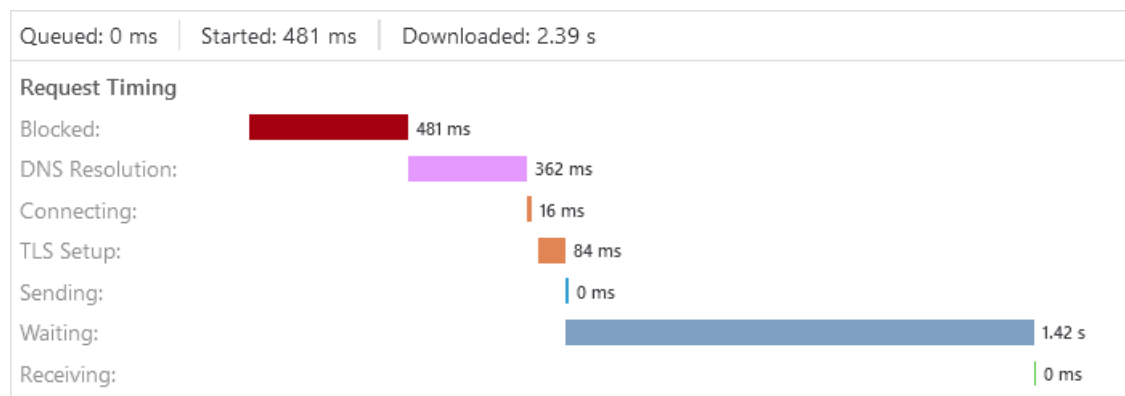


Figure 9 Request Timing from out of PSTU Network (Before Deploying SIISDM)

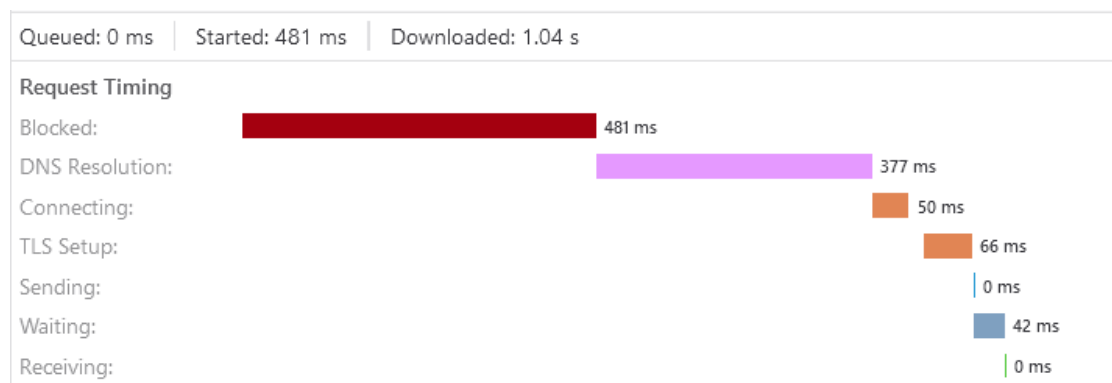


Figure 10 Request Timing from out of PSTU Network (After Deploying SIISDM)

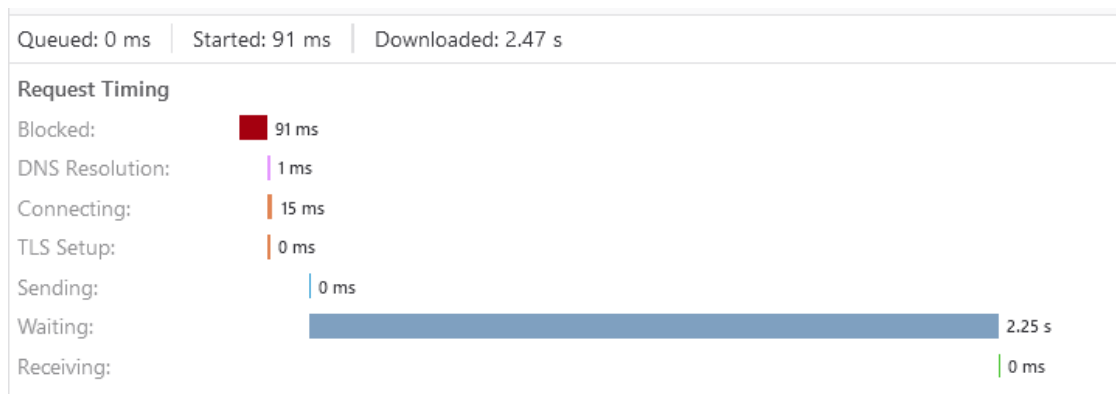


Figure 11 Request Timing from PSTU Network (Before Deploying SIISDM)

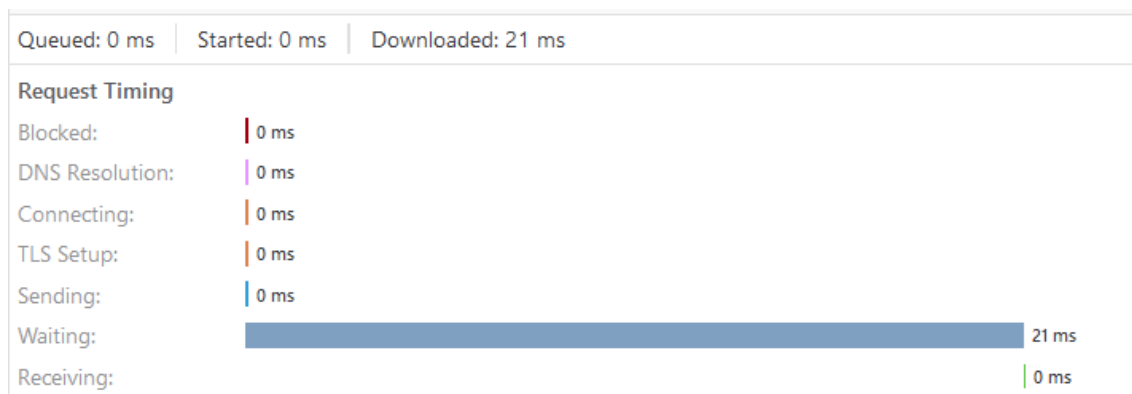


Figure 12 Request Timing from PSTU Network (After Deploying SIISDM)

By comparing waiting time in queue, request timing result ensure the increase of efficiency of the system.

4.4.2 Dynamic Rate Limit Configuration

Another identified issue involved the static nature of rate limiting configurations within security constraints. These configurations were set via service configuration and DI as singletons, hindering dynamic adjustments based on user roles, request types, or accessed data. To address this, SIISDM facilitated the extension of the dependency injection scope for rate limiting configurations to "scoped," allowing for dynamic adjustments based on individual users. This improved system flexibility and adaptability in managing security measures.

Comparing the environment variable configurations, the selection of the most appropriate mathematical formulation for a rate limiting constraint depends on the desired behavior. Fixed rate limiting, while simple to implement, lacks flexibility in handling burst traffic. The leaky bucket model offers more flexibility but can drop requests during excessive bursts. The token bucket model provides greater control over bursts within defined limits, while the sliding window approach focuses on maintaining a sustainable rate within a

specific time window. SIISDM's dynamic rate limiting approach offers a more tailored solution by allowing for rate adjustments based on user roles, request types, and data access patterns, providing enhanced security and performance.

In our experiment on rate limiting implementation, we have following rate limit Scenario traced from the response header of the application. Here applied dynamic policy and prompt multiple parallel connections at same time from same client device targeting exceed up to current maximum allowable requests per second.

```
HTTP/2 200
content-type: text/html; charset=utf-8
server: Microsoft-IIS/10.0
strict-transport-security: max-age=0; includeSubDomains
date: Wed, 26 May 2024 06:33:41 GMT
X-Firefox-Spdy: h2
```

Figure 13 Response Header (Before Rate Limit Implementation)

```
HTTP/2 200
content-type: text/html; charset=utf-8
server: Microsoft-IIS/10.0
strict-transport-security: max-age=2592000
x-rate-limit-limit: 1h
x-rate-limit-remaining: 1997
x-rate-limit-reset: 2024-05-26T07:43:29.8091501Z
date: Wed, 26 May 2024 06:45:59 GMT
X-Firefox-Spdy: h2
```

Figure 14 Response Header (After Rate Limit Implementation)

```
HTTP/2 429
content-type: text/plain
retry-after: 3443
server: Microsoft-IIS/10.0
x-powered-by: ASP.NET
date: Wed, 26 May 2024 06:46:06 GMT
X-Firefox-Spdy: h2
```

Figure 15 Response Header (After API calls quota exceeded)

Thus ensure the enhanced security of API endpoint accessibility by throttling excessive requests, rate limiting safeguards applications from DoS attacks and malicious attempts to overwhelm resources.

The successful implementation of SIISDM on these live applications demonstrates its effectiveness in enhancing security by identifying and mitigating vulnerabilities that could have significant negative impacts. The model's ability to ensure proper configuration management and dynamic security adjustments contributes to a more secure and efficient information system environment. By proactively integrating security considerations throughout the Software Development Lifecycle (SDLC), SIISDM enables early identification and remediation of vulnerabilities, reducing the risk of exploitation and minimizing potential costs associated with data breaches. Moreover, a strong security posture enhances an institution's reputation, fostering trust among stakeholders and partners. Additionally, SIISDM can help institutions achieve compliance with various security regulations and standards, demonstrating a commitment to responsible data management and protection.

CHAPTER V

CONCLUSION AND FUTURE WORKS

5.1 Introduction

Security assurance models offer a structured and comprehensive methodology for assessing and fortifying the security posture of information systems. Understanding their core principles, functionalities, and limitations is essential for researchers, practitioners, and policymakers seeking to develop and implement effective strategies for building trustworthy and resilient institutional information systems in the digital age.

5.2 Key Contributions

By this research study, our work is emphasize effective security consideration of modern secure institute information system.

Here enlisted key contributions are:

- Have prepared a dataset about current situation of security concern of institute information systems over Asian sub-continent through survey response.
- Apply security constraint to find the appropriate solutions of security issues on development phases.
- Prepared a security assurance model for testing readiness of developing institutional information system following SDL and ISO/IEC 27001:2022(E).
- Impalement the model to mitigate critical security issues during development and experimented over deployed system applications.

Moreover, for the developing countries like Bangladesh, the sustainable development goals SDG introduced by United Nation involves Industry, Innovation and Infrastructure within the target to build resilient infrastructure, promote sustainable industrialization and foster innovation. Under this aim, SDG support domestic technology development, research and innovation including the initiative for significantly increase access to information and communications technology and strive to provide universal and affordable access. Thus the research task will be a forward step to implement the SDG targets through ensuring industrial secure information system. From this viewpoint, this research have aim to develop the SSA model that may be very helpful to the software

industries and their Clients and Partners including public sectors and large scale government projects in terms of ICT expertise and practices.

5.3 Further Research Scopes

Hence by the study, we tried our best to come-up with an appropriate solution in a targeted security assurance model of institute information system, our contribution shows state of art. But this study not limited to extend further researches. Here enlisted some issues that can be introduced to researchers for study in future.

Self-Tuning Mechanisms and Threat Intelligence Integration

Developing self-tuning rate limits that learn and adapt over time presents a valuable research area. These systems could automatically adjust limits based on observed attack patterns and historical data, improving effectiveness and reducing the burden of manual intervention. Additionally, integrating rate limiting configurations with threat intelligence feeds offers a proactive approach. Real-time updates on known malicious actors and suspicious IP addresses would enable preemptive adjustments to rate limits, mitigating emerging threats before they can exploit vulnerabilities.

Standardization and User Experience Considerations

Standardized approaches for configuring rate limits across different systems could significantly simplify management and enhance consistency. Research could explore developing standardized configuration formats or automated tools that streamline the process for administrators.

Balancing security with user experience is paramount. Research should investigate effective ways to implement rate limits that minimize negative impacts on legitimate users while effectively deterring malicious activity. This might involve exploring alternative throttling mechanisms or providing informative feedback to users encountering rate limits.

By delving deeper into these research areas, the field of rate limiting configuration can evolve beyond static thresholds. Dynamic, context-aware, and self-tuning approaches informed by threat intelligence hold the key to developing more sophisticated and effective rate limiting strategies. This will ultimately lead to a more secure and robust digital ecosystem for information systems.

In other words, the findings presented in this research offer a valuable roadmap for institutions seeking to elevate the security of their information systems. The constructed

dataset provides a starting point for understanding regional security concerns, while the SIISDM offers a practical framework for implementing robust security practices throughout the development lifecycle. Future research can explore further refinements to the SIISDM and its broader application across diverse institutional contexts. By continuing to prioritize security considerations, institutions can create a more secure and trustworthy digital environment for their information systems.

References

1. Al-Harrasi, A., Shaikh, A. K., & Al-Badi, A. (2023). Towards protecting organisations' data by preventing data theft by malicious insiders. *International Journal of Organizational Analysis*, 31(3), 875–888. <https://doi.org/10.1108/IJOA-01-2021-2598/FULL/PDF>
2. Aljabri, M., Aldossary, M., Al-Homeed, N., Alhetelah, B., Althubiany, M., Alotaibi, O., & Alsaqer, S. (2022). Testing and exploiting tools to improve owasp top ten security vulnerabilities detection. *2022 14th International Conference on Computational Intelligence and Communication Networks (CICN)*, 797–803.
3. Alkhatib, M. (2022). An Effective and Secure Quality Assurance System for a Computer Science Program. *Computer Systems Science and Engineering*, 41(3), 975–995. <https://doi.org/10.32604/csse.2022.021398>
4. Alnaseef, F., Niazi, M., Mahmood, S., Alshayeb, M., & Ahmad, I. (2023). Towards a successful secure software acquisition. *Information and Software Technology*, 164, 107315. <https://doi.org/10.1016/j.infsof.2023.107315>
5. Barkatullah, T. M., Rahman, T., & Hussain, M. F. (2023). *Bangladesh Cyber Threat Landscape* 2022. https://ictd.portal.gov.bd/sites/default/files/files/ictd.portal.gov.bd/publications/effc311d_5097_46ba_afa4_5f44b60a93e6/Bangladesh%20Cyber%20Threat%20Landscape%202022.pdf
6. Bauer, A., Coppola, R., Alégroth, E., & Gorschek, T. (2023). Code review guidelines for GUI-based testing artifacts. *Information and Software Technology*, 163, 107299. <https://doi.org/10.1016/J.INFSOF.2023.107299>
7. Bian, C. (2023). Data as Assets in Foreign Direct Investment: Is China's National Data Governance Compatible with its International Investment Agreements? *Asian Journal of International Law*, 13(2), 342–364. <https://doi.org/10.1017/S2044251322000595>
8. Ceberio, M., & Kreinovich, V. (2014). Constraint Programming and Decision Making. In *Studies in Computational Intelligence* (Vol. 539). Springer International Publishing. <https://link.springer.com/10.1007/978-3-319-04280-0>

9. Dash, B., & Ansari, M. F. (2022). *An effective cybersecurity awareness training model: first defense of an organizational security strategy*.
10. Frazzetto, D., Nielsen, T. D., Pedersen, T. B., & Šikšnys, L. (2019). Prescriptive analytics: a survey of emerging trends and technologies. *VLDB Journal*, 28(4), 575–595. <https://doi.org/10.1007/S00778-019-00539-Y/FIGURES/4>
11. Gao, X., Gong, S., Wang, Y., Wang, X., & Qiu, M. (2022). An economic analysis of information security decisions with mandatory security standards in resource sharing environments. *Expert Systems with Applications*, 206, 117894. <https://doi.org/10.1016/j.eswa.2022.117894>
12. Gluck, D., & Mazzoli, R. (2024). *Microsoft Security Development Lifecycle (SDL) - Microsoft Service Assurance*. Microsoft Compliance. <https://learn.microsoft.com/en-us/compliance/assurance/assurance-microsoft-security-development-lifecycle>
13. Gurcan, F., Dalveren, G. G. M., Cagiltay, N. E., Roman, D., & Soylu, A. (2022). Evolution of Software Testing Strategies and Trends: Semantic Content Analysis of Software Research Corpus of the Last 40 Years. *IEEE Access*, 10, 106093–106109. <https://doi.org/10.1109/ACCESS.2022.3211949>
14. International Organization for Standardization (ISO). (2022). *Information technology — Security techniques — Information security management systems — Requirements*. <https://www.iso.org/standard/27001>
15. Keskin, O. F., Caramancion, K. M., Tatar, I., Raza, O., & Tatar, U. (2021). Cyber Third-Party Risk Management: A Comparison of Non-Intrusive Risk Scoring Reports. *Electronics*, 10(10), 1168. <https://doi.org/10.3390/electronics10101168>
16. Khalil, S. M., Bahsi, H., Dola, H. O., Korõtko, T., McLaughlin, K., & Kotkas, V. (2023). Threat Modeling of Cyber-Physical Systems - A Case Study of a Microgrid System. *Computers & Security*, 124, 102950. <https://doi.org/10.1016/j.cose.2022.102950>
17. Khan, R. A., Khan, S. U., Alzahrani, M., & Ilyas, M. (2022). Security Assurance Model of Software Development for Global Software Development Vendors. *IEEE Access*, 10, 58458–58487. <https://doi.org/10.1109/ACCESS.2022.3178301>

18. Khan, R. A., Khan, S. U., Khan, H. U., & Ilyas, M. (2022). Systematic Literature Review on Security Risks and its Practices in Secure Software Development. *IEEE Access*, 10, 5456–5481. <https://doi.org/10.1109/ACCESS.2022.3140181>
19. Kizza, J. M. (2024). *Access Control and Authorization*. 195–214. https://doi.org/10.1007/978-3-031-47549-8_9
20. Lepenioti, K., Bousdekis, A., Apostolou, D., & Mentzas, G. (2020). Prescriptive analytics: Literature review and research challenges. *International Journal of Information Management*, 50, 57–70. <https://doi.org/10.1016/J.IJINFOMGT.2019.04.003>
21. Manthiramoorthy, C., Mohamed, K., Khan, S., & Ameen, N. (2024). Comparing Several Encrypted Cloud Storage Platforms. *International Journal of Mathematics, Statistics, and Computer Science*, 2, 44–62. <https://doi.org/10.59543/IJMSCS.V2I.7971>
22. McGraw, G. (2006). *Software Security: Building Security In*. Addison-Wesley Professional.
23. Meng, L., Zhang, C., Ren, Y., Zhang, B., & Lv, C. (2020). Mixed-integer linear programming and constraint programming formulations for solving distributed flexible job shop scheduling problem. *Computers & Industrial Engineering*, 142, 106347.
24. MITRE Corporation. (2023). Record of CVE-2023-29552. In *National Vulnerability Database*. National Institute of Standards and Technology (NIST). <https://nvd.nist.gov/vuln/detail/CVE-2023-29552>
25. Nandakumar, K., Vinod, V., Akbar Batcha, S. M., Sharma, D. K., Elangovan, M., Poonia, A., Mudlappa Basavaraju, S., Dogiwal, S. R., Dadheech, P., & Sengan, S. (2021). Securing data in transit using data-in-transit defender architecture for cloud communication. *Soft Computing*, 25(18), 12343–12356. <https://doi.org/10.1007/S00500-021-05928-6/FIGURES/9>
26. Pal, S. K., & Sarker, P. C. (2023). SMART Bangladesh vision 2041: concept of a sustainable developed country. *Environmental Management and Sustainable Development*, 12(1), 67–81.

27. Pargaonkar, S. (2023). Advancements in security testing: A comprehensive review of methodologies and emerging trends in software quality engineering. *International Journal of Science and Research (IJSR)*, 12(9), 61–66.
28. Pierre-Alain, Y., & Laurent, Z. (2021). Integration of Constraint Programming and Model-Based Approach for System Synthesis. *15th Annual IEEE International Systems Conference, SysCon 2021 - Proceedings*. <https://doi.org/10.1109/SYSCON48628.2021.9447096>
29. Ponsard, C., Deprez, J.-C., & Darimont, R. (2020). Formalizing security and safety requirements by mapping attack-fault trees on obstacle models with constraint programming semantics. *2020 IEEE Workshop on Formal Requirements (FORMREQ)*, 8–13.
30. Quran, *The Holy Quran*. (n.d.).
31. Raiyat Aliabadi, M., Seltzer, M., Vahidi Asl, M., & Ghavamizadeh, R. (2021). ARTINALI#: An Efficient Intrusion Detection Technique for Resource-Constrained Cyber-Physical Systems. *International Journal of Critical Infrastructure Protection*, 33, 100430. <https://doi.org/10.1016/J.IJCIP.2021.100430>
32. Rajapaksha, S., Senanayake, J., Kalutarage, H., & Al-Kadri, M. O. (2024). Enhancing Security Assurance in Software Development: AI-Based Vulnerable Code Detection with Static Analysis (pp. 341–356). https://doi.org/10.1007/978-3-031-54129-2_20
33. Reveraert, M., Sas, M., Reniers, G., Hardyns, W., & Sauer, T. (2024). Insider threats to critical infrastructure: A typology. *Management and Engineering of Critical Infrastructures*, 53–83. <https://doi.org/10.1016/B978-0-323-99330-2.00012-X>
34. Saiful Alam Khan, Engr. M., Farhad Hussain, Tawhidur Rahman, & Aminul Islam, K. (2024). Guidelines on Information Security Practices for Public and Government Entities. In *Bangladesh Government's e-Government Computer Incident Response Team*. <https://www.cirt.gov.bd/information-security-practices-public-and-government/>

35. Spiekermann, S. (2021). Value-based Engineering: Prinzipien und Motivation für bessere IT-Systeme. *Informatik Spektrum*, 44(4), 247–256. <https://doi.org/10.1007/s00287-021-01378-4>
36. Spiekermann, S., Krasnova, H., Hinz, O., Baumann, A., Benlian, A., Gimpel, H., Heimbach, I., Köster, A., Maedche, A., Niehaves, B., Risius, M., & Trenz, M. (2022). Values and Ethics in Information Systems: A State-of-the-Art Analysis and Avenues for Future Research. *Business & Information Systems Engineering*, 64(2), 247–264. <https://doi.org/10.1007/s12599-021-00734-8>
37. Umeugo, W. (2023). SECURE SOFTWARE DEVELOPMENT LIFECYCLE: A CASE FOR ADOPTION IN SOFTWARE SMES. *International Journal of Advanced Research in Computer Science*, 14(1).
38. Varela-Vaca, A. J., & Gasca, R. M. (2013). Towards the automatic and optimal selection of risk treatments for business processes using a constraint programming approach. *Information and Software Technology*, 55(11), 1948–1973.
39. Wen, S.-F., & Katt, B. (2023). A quantitative security evaluation and analysis model for web applications based on OWASP application security verification standard. *Computers & Security*, 135, 103532. <https://doi.org/10.1016/j.cose.2023.103532>
40. Wen, S.-F., & Katt, B. (2024). Exploring the role of assurance context in system security assurance evaluation: a conceptual model. *Information & Computer Security*, 32(2), 159–178.
41. Zahan, N., Shohan, S., Harris, D., & Williams, L. (2023). Do Software Security Practices Yield Fewer Vulnerabilities? 2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), 292–303. <https://doi.org/10.1109/ICSE-SEIP58684.2023.00032>

APPENDICES

Appendix A

Enlisted Security issues for Institute Information System

Category	Security Issue	Risk Level	Description
Data Security	Data Breach	High	Unauthorized access to or disclosure of sensitive data. (e.g., Hacking, insider threats, accidental data leaks)
	Data Loss	High	Permanent loss of access to critical data due to hardware failure, natural disaster, or other factors. (e.g., Hard drive failure, ransomware attacks)
	Data Tampering	Medium	Unauthorized modification of data, resulting in inaccurate or misleading information. (e.g., Man-in-the-middle attacks, malware injection)
	Unencrypted Data	Medium	Sensitive data stored or transmitted without encryption, making it vulnerable to interception. (e.g., Unencrypted email attachments, lack of disk encryption)
Access Control	Unauthorized Access	High	Gaining access to a system or data without proper authorization. (e.g., Weak passwords, privilege escalation attacks)
	Least Privilege Not Enforced	Medium	Users granted more access than necessary to perform their duties. (e.g., Overly permissive user permissions, default administrator accounts)
	Account Takeover	Medium	An attacker gains control of a legitimate user account. (e.g., Phishing attacks, credential stuffing)
Network Security	Denial-of-Service (DoS) Attack	High	Overwhelming a system with traffic, making it unavailable to legitimate users. (e.g., Distributed Denial-of-Service (DDoS) attacks, resource exhaustion attacks)

Category	Security Issue	Risk Level	Description
	Malware Infection	High	Malicious software that can steal data, disrupt operations, or cause damage. (e.g., Viruses, worms, ransomware)
	Man-in-the-Middle Attack	Medium	Attacker intercepts communication between two parties and can eavesdrop or modify data. (e.g., Unsecured Wi-Fi networks, ARP spoofing)
System Vulnerabilities	Unpatched Software	High	Outdated software with known vulnerabilities that attackers can exploit. (e.g., Failure to apply security patches in a timely manner)
	Misconfiguration	Medium	System or application settings not configured securely, introducing vulnerabilities. (e.g., Weak default configurations, insecure protocols enabled)
	Zero-Day Exploit	High	A newly discovered vulnerability for which no patch exists yet. (e.g., Exploiting vulnerabilities before a vendor releases a security fix)
Misconfiguration	Security Settings Not Enabled	Medium	Critical security features like firewalls or intrusion detection systems disabled or not properly configured. (e.g., Disabling firewalls for testing purposes)
	Default Passwords Not Changed	Medium	Leaving default passwords on devices or systems creates a known vulnerability. (e.g., Default router passwords)
Insider Threats	Disgruntled Employee	High	A disgruntled employee with authorized access intentionally damages or steals data. (e.g., Sabotage, data exfiltration)
	Accidental Insider Errors	Medium	Unintentional mistakes by authorized users that can lead to security breaches. (e.g., Sending sensitive data to the wrong recipient, falling for phishing attacks)

Category	Security Issue	Risk Level	Description
Social Engineering	Phishing	Medium	Deceptive emails or messages designed to trick users into revealing personal information or clicking malicious links. (e.g., Emails disguised as legitimate sources, fake websites)
	Pretexting	Medium	Impersonating a trusted individual or organization to gain access to confidential information. (e.g., Phishing attacks using social engineering tactics)
Physical Security	Uncontrolled Physical Access	Medium	Unauthorized access to physical locations housing IT equipment or sensitive data. (e.g., Lack of access control to server rooms, tailgating)
	Theft of Physical Devices	Medium	Loss or theft of laptops, mobile devices, or other IT equipment containing sensitive data. (e.g., Lost laptops, stolen smartphones)

Appendix B

Request Timing Records (SIISDM Deploying)

Accessing from outside of PSTU Network

Request Timing	Started Time		Process time		Leaving Time	
	Before	After	Before	After	Before	After
Blocked	0	0	481	481	481	481
DNS Resolution	481	481	362	377	843	858
Connecting	843	858	16	50	859	908
TLS setup	859	908	84	66	943	974
Sending	943	974	0	0	943	974
Waiting	943	974	1400	42	2343	1016
Receiving	2343	1016	0	0	2343	1016

Appendix C
Request Timing Records (SIISDM Deploying)
Accessing from inside of PSTU Network

Request Timing	Started Time		Process time		Leaving Time	
	Before	After	Before	After	Before	After
Blocked	0	0	91	0	91	0
DNS Resolution	91	0	1	0	92	0
Connecting	92	0	15	0	107	0
TLS setup	107	0	0	0	107	0
Sending	107	0	0	0	107	0
Waiting	107	0	2250	21	2357	21
Receiving	2357	21	0	0	2357	21

End of the Document